ESSnet Big Data

Specific Grant Agreement No 2 (SGA-2)

https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata
http://www.cros-portal.eu/

Framework Partnership Agreement Number **11104.2015.006-2015.720**

Specific Grant Agreement Number **11104.2016.010-2016.756**

Work Package 8

Methodology

Deliverable 8.3

Report describing the IT-infrastructure used and the accompanying processes developed and skills needed to study or produce Big Data based official statistics

Prepared by ESSNet Big Data WP8 members

Anke Consten, Valentin Chavdarov, Piet J.H. Daas, Vesna Horvat,
Jacek Maślankowski, Sónia Quaresma, Monica Scannapieco,
Magdalena Six, Tiziana Tuoto

ESSnet co-ordinator:

Peter Struijs (CBS, Netherlands)
p.struijs@cbs.nl
telephone : +31 45 570 7441
mobile phone : +31 6 5248 7775

Table of content

1. Introduction 4
2. Executive summary 7
3. List of issues 9
   3.1. Big Data processing life cycle 9
      3.1.1. Introduction 9
      3.1.2. Examples and methods 9
      3.1.3. Discussion 10
      3.1.4. Literature 11
   3.2. Metadata management (ontology) 11
      3.2.1. Introduction 11
      3.2.2. Examples and methods 12
      3.2.3. Discussion 14
      3.2.4. Literature 15
   3.3. Format of Big Data processing 15
      3.3.1. Introduction 15
      3.3.2. Examples and methods 15
      3.3.3. Discussion 18
      3.3.4. Literature 18
   3.4. Data-hub and Data-lake 18
      3.4.1. Introduction 18
      3.4.2. Examples and methods 18
      3.4.3. Discussion 19
      3.4.4. Literature 20
   3.5. Data source integration 20
      3.5.1. Introduction 20
      3.5.2. Examples and methods 20
      3.5.3. Discussion 24
      3.5.4. Literature 24
   3.6. Choosing the right infrastructure 25
      3.6.1. Introduction 25
      3.6.2. Examples and methods 26
      3.6.3. Discussion 30

3.6.4. Literature ... 31
3.7. List of secure and tested API's ... 31
3.7.1. Introduction ... 31
3.7.2. Examples and methods ... 31
3.7.3. Discussion ... 33
3.7.4. Literature ... 33
3.8. Shared libraries and documented standards ... 34
3.8.1. Introduction ... 34
3.8.2. Examples and methods ... 34
3.8.3. Discussion ... 36
3.8.4. Literature ... 37
3.9. Speed of algorithms ... 37
3.9.1. Introduction ... 37
3.9.2. Examples and methods ... 38
3.9.3. Discussion ... 40
3.9.4. Literature ... 40
3.10. Training/skills/knowledge ... 41
3.10.1. Introduction ... 41
3.10.2. Examples and methods ... 41
3.10.3. Discussion ... 43
3.10.4. Literature ... 43
4. Conclusions ... 44
5. Abbreviations and acronyms ... 45
6. List of figures and tables ... 46
7. Literature ... 47

1. Introduction

The overall objective of the ESSnet Big Data is to prepare the ESS for integration of big data sources into the production of official statistics. The award criteria mentioned that the project has to focus on running pilot projects exploring the potential of selected big data sources for producing or contributing to the production of official statistics. Aim of these pilots is to undertake concrete action in the domain of big data and obtain hands-on experience in the use of big data for official statistics.

A consortium of 22 partners, consisting of 20 National Statistical Institutes and 2 Statistical Authorities has been formed in September 2015 to meet the objectives of the project. According to the Framework Partnership Agreement (FPA) between the consortium and Eurostat, the project runs between February 2016 and May 2018. To concentrate the work as much as possible on the pilots, the consortium has organised its work around the pilots. More specifically, the consortium has subdivided its work into work packages (WP's). The work packages including their ultimate aims by the end of the project are listed in table 1.

Table 1: description of the work packages in the ESSnet Big Data Programme

Work Package	Description
WP1 Webscraping / Job Vacancies	This WP wants to demonstrate by concrete estimates which approaches (techniques, methodology etc.) are most suitable to produce statistical estimates in the domain of job vacancies and under which conditions these approaches can be used in the ESS. The intention is to explore a mix of sources including job portals, job adverts on enterprise websites, and job vacancy data from third party sources.
WP2 Webscraping / Enterprice Characteristics	This WP investigates which webscraping, text mining and inference techniques can be used to collect, process and improve general information about enterprises.
WP3 Smart Meters	This WP wants to demonstrate by concrete estimates whether buildings equipped with smart meters (= electricity meters which can be read from a distance and measure electricity consumption at a high frequency) can be used to produce energy statistics but can also be relevant as a supplement for other statistics e.g. census housing statistics, household costs, impact on environment, statistics about energy production.
WP4 AIS Data	The aim of this WP is to investigate whether real-time measurement data of ship positions (measured by the so-called AIS-system) can be used 1) to improve the quality and internal comparability of existing statistics and 2) for new statistical products relevant for the ESS.
WP5 Mobile Phone Data	The aim of this WP is to investigate how NSIs may obtain more or less 'stable' and continuous access to the data of mobile phone operators.
WP6 Early Estimates	The aim of this WP is to investigate how a combination of (early available) multiple Big Data sources and existing official statistical data can be used in order to create existing or new early estimates for statistics.

WP7 Multi Domains	The aim of this WP is to investigate how a combination of Big Data sources and existing official statistical data can be used to improve current statistics and create new statistics in various statistical domains.
WP8 Methodology	The aim of this work package is laying down a generally applicable foundation in areas such as methodology, quality and IT infrastructure for future use of the selected big data sources from the pilots within the European Statistical System.

As can be seen from table 1, the work packages 1 to 7 each deal with one pilot and a concrete output. The aim of WP 8 is to generalise the findings of the pilots in order to relate them to the conditions for future use of big data sources within the ESS.

Seven National Statistical Institutes participate in WP8: the national statistical institutes of Austria, Bulgaria, Italy, Poland, Portugal, Slovenia and The Netherlands (Work package leader).

WP8 results in four deliverables, of which this deliverable is the third one:

8.1 Literature overview (delivered on 31 December 2017, but is living document)

8.2 Report describing the quality aspects of Big Data (planned for 31 May 2018)

8.3 Report describing the IT-infrastructure used and the accompanying processes developed and skills needed to study or produce Big Data based official statistics (the current deliverable)

8.4 Report describing the methodology of using Big Data for official statistics (planned for 31 May 2018)

The topics (chapters) described in this report are based on the results of the WP 8 workshop held on the 25th and 26th of April 2017 at Statistics Netherlands. During these two days, a group of 18 people (representatives from a large number of the partners involved in the ESSnet Big Data, from as many WP's as possible, and from Eurostat) identified the main topics in the areas of Methodology, Quality and IT when using Big Data for official statistics in the context of WP 1-7 of the ESSnet on Big Data.

In this report main topics identified in the area of IT when using Big Data for official statistics are described.. When looked upon the issue of using Big Data for official statistics it is clear that the available IT infrastructure determines the way by which quality and methodological issues will be dealt with. The goal of this report is to expand on the topics identified and link these to the work performed in WP1 -7 of the ESSnet Big Data. It is also a valuable resource of information for anyone, statisticians in particular, interested in the essential IT-aspects of Big Data.

Each chapter in this report covers different aspects of Big Data and includes:

- Big Data processing life cycle
- Metadata management (ontology)
- Format of Big Data processing
- Data-hub and Data-lake
- Data source integration
- Choosing the right infrastructure
- List of secure and tested API's
- Shared libraries and documented standards

- Speed of algorithms
- Training/skills/knowledge

The report ends with overall conclusions on the main topics in the area of IT when using Big Data for official statistics.

2. Executive summary

According to the structure of the report, these are the main findings:

1. The aim of work package 8 is laying down a generally applicable foundation in areas such as methodology, quality and IT infrastructure for future use of the selected big data sources from the pilots within the European Statistical System.
2. Big Data process for official statistics can be characterized at four different stages: collect, process, analyze and disseminate, this sequence is mostly used by Big Data pilots such as traffic intensity, web scraping enterprise characteristics or AIS data.
3. There is no unified framework for Metadata Management. We can rely on Big Data Quality Framework by UNECE but for metadata purposes, better way is to apply Common Metadata Framework by GSBPM and GAMSO. Data quality is evaluated in Big Data using equivalent indicators.
4. Format of Big Data sources in official statistics is various but some common data files can be listed, such as semi-structured CSV files (the most common format) and JSON files (for storing processed data), structured relational databases and unstructured NoSQL-like databases.
5. Data-hubs and Data-lakes mostly refer to the environment created in Apache Hadoop cluster or NoSQL database. A good example is a sandbox created by UNECE for Big Data purposes. However no ESSNet Big Data WP used a data-lake. Data-hubs were used to access smart meter data by WP3.
6. Variety of Big Data sources makes it no easy to integrate them directly. Different data sources (e.g., structured vs. unstructured) use different processing techniques and as a result, it may be difficult to find an attribute to link them during processing phase.
7. The right infrastructure for Big Data processing must be characterized by the following attributes: linear scalability, high throughput, fault tolerance, auto recovery, programing language interfaces, high degree of parallelism and distributed data processing. In ESSNet Big Data the variety of platform used included Linux and Windows as a landing zone and Java, Python, R, Spark and SAS for processing purposes.
8. Several API's (Application Programming Interfaces) were used by ESSNet Big Data WP's: Twitter API, Facebook Graph API, Google Maps API, Google Custom Search API, Bing API, Guardian API and Copernicus Open Access Hub to access public available data.
9. There is a list of Github repositories that contain software to start with Big Data in official statistics. It includes: Awesome Official Statistics software, ONS (Office for National Statistics) UK Big Data team, ONS (Office for National Statistics) UK Data Science Campus and ESTP Big Data course software.
10. Using the right algorithm will allow to increase the speed of data processing. It is especially important when processing streaming data.
11. The best way to become a data scientist is to start with improving the skills in the area of negotiating with data owners, set and maintain infrastructure, data combining, checking and editing large amounts of data, analyzing large amounts of data, using statistical methods including machine learning and visualize the data. Most of this skills can be improved with ESTP courses – European Statistical Training Program.
12. The general conclusion of the report is that there is a common set of tools, methods and libraries used by ESSNet countries and shared, e.g., Python and R language, various API's and machine learning algorithms. Moreover, several Github repositories with statistical software

exist where countries can use and give feedback on different tools, such as software to detect the presence of an enterprise in social media or its ecommerce activitity.

3. List of issues

3.1. Big Data processing life cycle

3.1.1. Introduction

The Big Data processing life cycle focusses on the part of the Big Data process immediately following the generation and availability of Big Data. During the Big Data life cycle, the data is checked, transformed, aggregated etc. and -in the end- turned into information. This process is not linear and contains many cycles as findings at the end of the chain may initiate the need to improve some or a number of the previous steps (see below). Big Data can be processed in a batch but also in a streaming way (see section 2.5 and 2.6).

3.1.2. Examples and methods

No WP in the ESSnet Big Data has reached the stage that a complete Big Data process has been setup, however, but both WP2 and 4 are almost there (WP2 del 2.2; WP4 del 4.3). The only example of a Big Data based statistical process known to date is the Traffic Intensity statistics process of Statistics Netherlands (Stat. Neth., 2015; Puts et al., 2016). The general steps described below apply to each of these examples.

In general, a Big Data process is considered to be composed of 4 steps (NIST BD WG, 2015). In the Generic Statistical Business Process Model (GSBPM, 2013) these steps are identified as the Collect, Process, Analyse and Disseminate phases, respectively. The first two steps involve supporting technologies to acquire and store data and to process and prepare it for analysis. The last two steps refer to techniques used to analyze and extract information from Big Data and to disseminate the results. The results of each step may result in findings that require redoing some (part of) the previous steps; this indicates an evaluation at the end of each stage. Hence the use of term Big Data processing (life) cycle which ultimately results in the dissemination of the final product; a statistic. Each step in the process is briefly discussed below.

1) Collect

In this step it is all about getting access to the data and making sure it can be processed. This can be challenging as many Big Data sources are collected and maintained by private organizations. When the data is accessed this means that the data is either stored locally, e.g. inside the office, or that it is put in an environment where it can be subsequently studied. This could be “the cloud” or even in an environment at the site of the producer of the data. Privacy considerations highly affect this choice as does the overall size of the data. For instance, mobile phone data has been studied at the site of the mobile phone provider because of both these issues (Demunter and Seynaeve, 2017). Typical for this step is that the data is made available in its ‘raw’ (unprocessed) form. Although initial studies often start with small samples, the ultimate aim is usually to include large amounts of data, if not all.

2) Process

Step two is all about processing and preparing the data prior to analysis. Quality checking and ultimately quality improvement are the major concerns here; this is sometimes also referred to as data cleaning. It is sometimes also described as the start of ‘data mining’. In the case of annotation, additional information is added to (again) improve the quality of the data. The effect of some of the adjustments is not always immediately clear and it thus may require several iterations to found out

the optimal approach for a particular data set. Especially for huge amounts of data, this step benefits the most from the availability of a well performing Big Data specific IT environment (Section 2.6). Although the data may still be unstructured at this stage, as a consequence of the adjustments made, the data becomes more and more structured. Part of the preparation may be combining the cleaned data with other datasets and/or aggregated to particular groups or areas. Many challenges still have to be solved at this stage as many Big Data sources lack any information on the units included or are actually event based. This requires either profiling of the units in a data source or converting events to units. An example of the latter is combining the check-in and check-out location data of public transport chip card data to deduce the trips taken by the users. Aggregation is challenging when detailed information on the units is lacking. However, since a considerable number of Big Data sources contain information on the geographical area or time period covered, those could be used as alternative levels of aggregation.

3) Analyse

Since Big Data can be extremely noisy and heterogeneous, even after considerable processing and aggregation steps, modelling is a way to deal with these issues. In addition, the increased focus on prediction when using Big Data may require the need for a whole new range of analysis methods not (yet) common to official statistics. Examples of the latter are many of the Machine Learning based approaches (see the Methodology report of WP8). Here, the focus is usually on predictive accuracy and less on the construction of correct stochastic data models. This represents a considerable paradigm shift in the statistical community. Findings observed at this stage may require the need to adjust some of the earlier steps to improve the quality of the data and its findings. Ultimate goal of this stage is to produce new insights, i.e. knowledge.

4) Disseminate

This stage is all about producing value. When large amounts of data have been used, this step will most likely also include a visualization with the aim to make the data better understandable for analysts and for those to which the findings are presented. A whole range of visualizations specific for huge amounts of data have and are currently being developed. Other products are reports and publications. The output of this step can also be used as input for other processes.

3.1.3. Discussion

Based on the experiences obtained in the ESSnet Big Data, the Traffic Intensity statistics example and the GSBPM, the most important steps in the Big Data processing life cycle have been described. Since adjustments made in one of the steps earlier in the process affect the findings in any of the subsequent steps, an iterative approach is usually applied, hence the Big Data processing life *cycle*. This indicates an evaluation at the end of each step. The main cause of this is the fact that a Big Data process is constructed in a data driven way, i.e. from input to output. This in contrast to the development of traditional statistical process which follows the reversed order; from output to input (GSBPM 2013; Design phase). In WP2 and also in WP4 the input derived way of working can be clearly observed in the sequence of the topics being studied and described (WP2 Del 2.2; WP4 Del 4.3). This was also the way in which the Traffic Intensity statistics was constructed in the Netherlands (Puts et al., 2016).

Managing the Big Data processing steps in an efficient way, both from a time and a costs perspective, helps to speed up the whole processing cycle. Because of their interdependency and the order of development (from input to output), it is a challenge to optimizes these data intensive steps in the most efficient way. This is why this part of the process is often considered more of an art and less of a scientific challenge (Peng and Matsui, 2016). In the beginning of chapter 2, Peng and Matsui (2016) state the following when discussing the data analysis topic:

> "*To the uninitiated, a data analysis may appear to follow a linear, one-step-after-the-other process which at the end, arrives at a nicely packaged and coherent result. In reality, data analysis is a highly iterative and non-linear process, better reflected by a series of epicycles …., in which information is learned at each step, which then informs whether (and how) to refine, and redo, the step that was just performed, or whether (and how) to proceed to the next step*."

This quote is applicable to the each of the four steps and the entire processing life cycle described above. The sharing of knowledge by people with experience in this area will certainly help to make the development of a Big Data process more transparent and somewhat more straight forward. Reaching this stage is not easy as it is highly dependent of each decision made along the way. The fact that chapter 3 of Deliverable 2.2 of WP2 includes a description of the first three steps of the Big Data processing life cycle (WP2 Del 2.2) illustrates the progress made in the ESSnet Big Data.

3.1.4. Literature

[1] Demunter, C., Seynaeve, G. (2017) Better quality of mobile phone data based statistics through the use of signaling information – the case of tourism statistics, NTTS Conference, 13-17 March 2017.

[2] GSPBM (2013) Generic Statistical Business Processing Model version 5.0. Located at: https://statswiki.unece.org/display/GSBPM/GSBPM+v5.0

[3] NIST Big Data Public Working Group (2015) NIST Big Data Interoperability Framework: Volume 1, Definitions. Final version. http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1500-1.pdf

[4] Peng, R., Matsui, E. (2016) The Art of Data Science, A guide for anyone who works with data. Leanpub book, lulu.com

[5] Puts, M., Tennekes, M., Daas, P.J.H., de Blois, C. (2016) Using huge amounts of road sensor data for official statistics. Paper for the European Conference on Quality in Official Statistics 2016, Madrid, Spain.

[6] Statistics Netherlands (2015) A first for Statistics Netherlands: launching statistics based on Big Data. Located at: https://www.cbs.nl/NR/rdonlyres/4E3C7500-03EB-4C54-8A0A-753C017165F2/0/afirstforlaunchingstatisticsbasedonbigdata.pdf.

[7] WP2 Del 2.2 (2017) Methodological and IT issues and Solutions. Chapter 3 "Description of a Reference Framework for Web Scraping of Enterprises Web Sites". Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/6/66/WP2_Deliverable_2.2_2017_07_31.pdf

[8] WP4 Del 4.3 (2018) Report about sea traffic analysis using AIS-data. Located at; https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/5/5c/WP4_Deliverable_4.3_2017_07_21_v1.0.pdf

3.2. Metadata management (ontology)

3.2.1. Introduction

It is important to have (high quality) metadata available for Big Data. This is essential for nearly all uses of Big Data. The level of specification and formalization of metadata can vary from metadata written in natural language with a documentation purpose to metadata formally specified through machine-

readable languages. For instance, ontologies belong to this latter category, by formal specifying concepts, properties and constraints over modelled business domains.

Typically, metadata can be managed in three different ways – active, semi-active and passive. Active management is when metadata is incorporated in the data set, passive is when metadata store is external to the data set and semi-active is an hybrid – some metadata are managed as passive, other as active. For instance, mobile phone data metadata are in the repository (structural metadata) which is an example of active metadata management. For text from websites, structural metadata are not provided so they can be classified as passive metadata management.

Data quality can be evaluated based on three hyperdimensions, depending on the entity being processed. It includes data, metadata and source [1]. In this chapter we will concentrate on metadata, including its quality and management issues.

3.2.2. Examples and methods

There are several frameworks that follow the rules of metadata management. Among them, there are GSBPM and GAMSO; both use GSIM as the common framework for metadata management [2]. In Table 1. there is a set of core principles regarding the metadata management.

Table 1. Core principles of metadata management of the Common Metadata Framework

Group	Principle
Metadata handling	***Statistical Business Process Model***: Manage metadata with a focus on the overall statistical business process model.
	Active not passive: Make metadata active to the greatest extent possible. Active metadata are metadata that drive other processes and actions. Treating metadata this way will ensure they are accurate and up-to-date.
	Reuse: Reuse metadata where possible for statistical integration as well as efficiency reasons
	Versions: Preserve history (old versions) of metadata.
Metadata Authority	***Registration***: Ensure the registration process (workflow) associated with each metadata element is well documented so there is clear identification of ownership, approval status, date of operation, etc.
	Single source: Ensure that a single, authoritative source ('registration authority') for each metadata element exists.
	One entry/update: Minimise errors by entering once and updating in one place.
	Standards variations: Ensure that variations from standards are tightly managed/approved, documented and visible.
Relationship to Statistical Cycle /Processes	***Integrity***: Make metadata-related work an integral part of business processes across the organisation.
	Matching metadata: Ensure that metadata presented to the end-users match the metadata that drove the business process or were created during the process.
	Describe flow: Describe metadata flow with the statistical and business processes (alongside the data flow and business logic).
	Capture at source: Capture metadata at their source, preferably automatically as a by-product of other processes.
	Exchange and use: Exchange metadata and use them for informing both computer based processes and human interpretation. The infrastructure for exchange of data and associated metadata should be based on loosely coupled components, with a choice of standard exchange languages, such as XML.
Users	***Identify users***: Ensure that users are clearly identified for all metadata processes, and that all metadata capturing will create value for them.

Group	Principle
	Different formats: The diversity of metadata is recognised and there are different views corresponding to the different uses of the data. Different users require different levels of detail. Metadata appear in different formats depending on the processes and goals for which they are produced and used.
	Availability: Ensure that metadata are readily available and useable in the context of the users' information needs (whether an internal or external user).

Source: https://statswiki.unece.org/display/GSBPM/Issue+%2322%3A+Metadata+Management+-+GSBPM+and+GAMSO [as of 5.01.2018].

As shown in the table above, metadata management can be grouped in four different categories regarding– metadata handling, metadata authority, relationship to statistical cycle/process and users. For each principle, a common set of indicators could be constructed, depending on the project being implemented.

As written in the introduction, metadata may be managed in three different ways – active, semi-active and passive. Depending on the metadata store that must be implemented, we can observe that most of the metadata management is performed in passive way, which means that metadata repository is stored in external information system. This allows managing metadata for the whole statistical data and allows better integration of the data.

Dealing with metadata management is important to increase the reliability and clearance of the data being processed. Because data quality is a part of the WP8 Quality Report, we will concentrate how to manage and deal with the quality of the metadata. In Table 2. we included metadata quality issues that should be regarded when collecting and processing the information.

Table 2. Metadata quality evaluation on WP4 AIS Data

Quality Dimension	Factors to consider	WP4 AIS Data
Complexity	Technical constraints Whether structured or unstructured Readability Presence of hierarchies and nesting	Data is stored in the NMEA data format. To decode the messages, libraries can be used that are available for different programming languages The data is highly structured Data is not human readable, although well readable by machines. No nesting available, but different record types
Completeness	Whether the metadata is available, interpretable and complete	Available, but one has to search the internet for it. Elements that are not processed automatically have to be put in by owner of the ship, who might not always be aware of all the details
Usability	Resources required to import and analyse Risk analysis	When dealing with Europe wide data for a longer time span, big data infrastructure and skills are needed Not applicable. These skills and infrastructure are also necessary for other sources

Quality Dimension	Factors to consider	WP4 AIS Data
Time-related factors	Timeliness Periodicity Changes through time	The data is high velocity. High quality data can be available in an instance Collection time and reference period is about the same. Not applicable. The data is collected and stored as is.
Linkability	Presence and quality of linking variables Linking level	Due to transmission errors, the IMO number necessary to link to maritime statistics are sometimes wrongly received. We developed a method to deal with this. Linking can be at vessel level based on the IMO number and precise geographical locations
Coherence - consistency	Standardisation Metadata available for key variables (classification variables, construct being measured)	Standardized Yes, metadata available
Validity	Transparency of methods and processes Soundness of methods and processes	DZ pre-processes the data, but there is no documentation available on how they do this. unknown

Source: A Suggested Framework for the Quality of Big Data, Deliverables of the UNECE Big Data Quality Task Team, December, 2014, https://statswiki.unece.org/display/bigdata/2014+Project?preview=%2F108102944%2F108298642%2FBig+Data+Quality+Framework+-+final-+Jan08-2015.pdf [as of 5.01.2018], WP4 data.

Every factor listed in table above can have a set of indicators. It is important to know that selected factors may not be relevant for various datasets, so the decision of using specific set of indicators may depend on the data source type (e.g., structured or unstructured) and its origin (e.g., administrative data source or web data).

3.2.3. Discussion

Metadata management is strictly related to data processing. However it is not possible to ensure accurate data management without knowing the quality dimension of the metadata. Having a reliable metadata management framework may lead to the clearance of the results of Big Data analysis.

As mentioned above, there is no unified framework for metadata management for Big Data purposes. Therefore, we have to select and provide a metadata management framework that best fits to the rules of official statistics. This is the reason why we decided to suggest to apply the rules and principles from well-known standards for statistical data processing.

However, the unstructured nature of several Big Data sources poses the serious issue to deal with metadata extraction. As an example, when scraping texts from Enterprises Web Sites (see WP 2), it is relevant to “represent” text in a more structured way, e.g. a term-document matrix in a bag-of-words approach or, alternatively, an embedding space or, even, derive an ontology for accessing data in a more structured way.

Nevertheless, every new Big Data project may use a common set of rules for metadata management. The principles of metadata management may be modified, included or excluded, depending on the characteristic of the metadata used for the specific project.

3.2.4. Literature

[1] https://statswiki.unece.org/display/GSBPM/Issue+%2322%3A+Metadata+Management+-+GSBPM+and+GAMSO [as of 5.01.2018].

[2] A Suggested Framework for the Quality of Big Data, Deliverables of the UNECE Big Data Quality Task Team, December, 2014, https://statswiki.unece.org/display/bigdata/2014+Project?preview=%2F108102944%2F108298642%2FBig+Data+Quality+Framework+-+final-+Jan08-2015.pdf [as of 5.01.2018].

3.3. Format of Big Data processing

3.3.1. Introduction

Processing large amounts of data in a reliable and efficient way introduces the need for a unified framework of languages and libraries. The variety of the tools used for data processing makes the decision of choosing the framework for Big Data processing very difficult. Typically, we can say that most of the data processing issues are covered in tools such as Apache Hadoop, Spark, Flink, Storm or Kafka. The decision of applying one of these tools can be made based on the criteria presented in chapter 2.3.2.

Let's consider processing large amount of data from mobile phone operators which are Mobile Call Records (MCR). You receive the data that is well structured however there are billions of rows every month. You can process this information with traditional relational database but the processing task will not be efficient. Then you can optimize your database by partitioning but still it will not be efficient. If the performance of the data processing is not an issue, you can still use this environment. But if you want to have a real time analysis you should use one of the software presented in this chapter.

We can divide the format of Big Data processing by the type of data (batch, streaming), the type of algorithm (e.g., MapReduce) and the type of method (e.g., Text Mining, Data Mining).

3.3.2. Examples and methods

When we think about Big Data processing, usually we are thinking of classical MapReduce paradigm that was incorporated in the Apache Hadoop project. It is used to process batch datasets, which means that they will not change. In Table 3. we presented the set of tools with the short characteristics of the way the data is processed.

Table 3. Main features of data processing by selected Big Data software

No.	Name	Link	Data processing characteristics
1	Apache Hadoop	http://hadoop.apache.org	Classic Big Data tool that use MapReduce as a processing paradigm, should be used to process large batch data sets that can be split into smaller parts; data stored in HDFS.
2	Apache Spark	http://spark.apache.org	In contrast to the Hadoop, all the data is processed in memory, must use external storage (e.g., filesystem, HDFS, databases); native language Scala, scripts can also be written in Python and execute with pyspark.
3	Apache Flink	http://flink.apache.org	Use for streaming data processed with Scala and Java, static data can be processed in Python, include machine learning libraries.
4	Apache Storm	http://storm.apache.org	Process unbounded streams of data, any programming language can be used for realtime analytics, online machine learning, ETL, etc. Databases can be used as additional data source.
5	Apache Kafka	http://kafka.apache.org	Used to build real-time data pipelines and streaming apps that are using stream processors. Applications are horizontally scalable (read more in chapter 2.6).

Source: http://apache.org [as of 20.12.2017].

As written in the table above in the data processing column, the way of data processing relies on the tool and data used. There are two main types of data that will direct us to the format of Big Data processing. These types are batch and streaming data. Depending on the data type, a suitable algorithm of Big Data processing will be used by the software. One of the efficient algorithm to process large datasets in efficient way is MapReduce.

MapReduce paradigm consists of two steps: the map and the reduce. The map is responsible for transforming input data row to the output list of keys and values:

```
map(key1,value) -> list<key2,value2>
```

The reduce is providing a new list of reduced output:

```
reduce(key2, list<value2>) -> list<value3>
```

The basic goal of the MapReduce paradigm is to process the similar objects only once. This is the reason why the similar objects are reduced.

All the experiments for work package 4 are programmed on Spark. Spark strongly relies on Maps, Reduces and Filters. As such, the map reduce paradigm can be used in spark and is used often in spark. As an example, in the job where ship intensities were calculated, one uses a map for collecting all the individual ship records per ship per grid cell and a reduce to aggregate all these records to come finally to a ship-count in the gridcell.

Before we made a decision of the format of Big Data processing we have to understand what type of data we have. Firstly, we have to start with the format of Big Data processing, like presented in Figure 1.

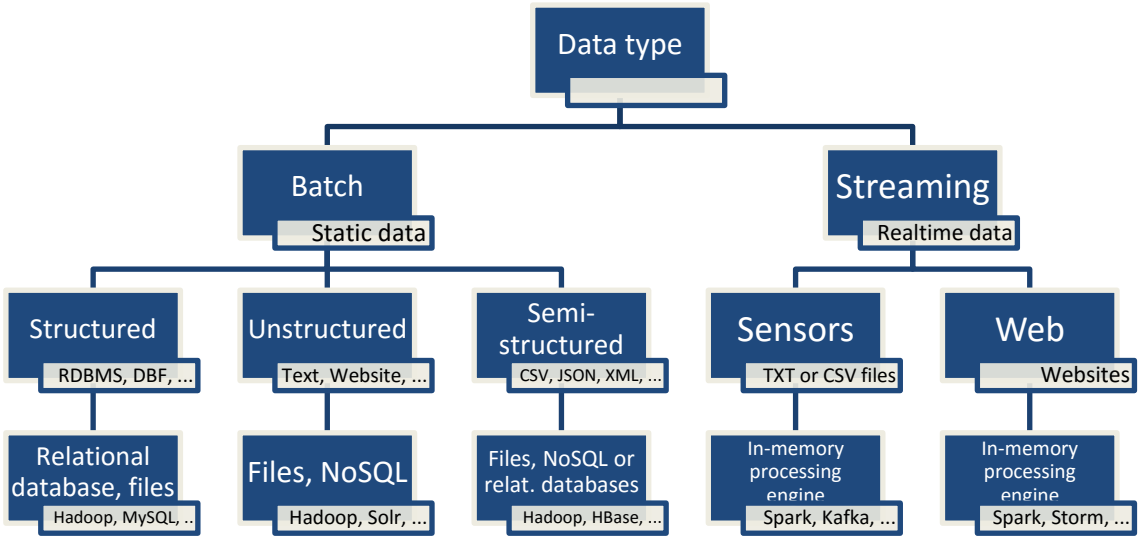


Figure 1. Decision process of using the format of data processing

According to the pilots conducted on Big Data by ESSNet countries we can say that most of the data is batch data, usually stored in CSV files, relational databases (MySQL) or NoSQL databases (Apache Solr). More information about the data storage tools is presented in chapter 2.6 of this report.

There are several different methods of data processing. Depending of the data type used the data can be processed by Text Mining, Web Mining (subclass of Text Mining to process web data), Natural Language Processing (one of Text Mining method), Data Mining or Machine Learning. These four methods have been used in pilots used in ESSNet on Big Data projects. The decision of the use of particular algorithm is presented in Table 4.

Table 4. Data processing examples depending on the data used

No.	Name	Data type	Libraries	Aim of the use	Used in
1	Data Mining	Structured	Pandas, Numpy (Python) Scala, RODBC (R)	Find patterns in the data, prediction	WP2,WP3, WP4,WP6, WP7
2	Text Mining	Unstructured – text	NLTK (Python)	Extract information from the data, Classification	WP1,WP2, WP7
3	Web Mining	Unstructured – websites	NLTK (Python)	Extract information from the web data, Classification	WP1,WP2
4	Natural Language Processing	Unstructured – text	NLTK (Python)	Stemming, Lemmatization, Tokenization, Extract information from the data (NLP is a part of Text Mining)	WP2,WP7
5	Machine Learning	Structured or Unstructured	Sklearn, Pandas, Numpy (Python) Ranger (R)	Sentiment analysis, supervised and unsupervised learning	WP1,WP2, WP3,WP4, WP6,WP7

Source: Own elaboration based on http://python.org [as of 20.12.2017].

As shown in the table above, different methods have been used by implementing pilots by ESSNet countries. For example, Twitter data were classified with supervised machine learning algorithms. Information from web data were extracted by web scraping techniques with Text Mining methods.

Natural Language Processing (in fact a part of Text Mining methods) was used to extract useful information from the text to prepare a good training dataset for machine learning purposes.

3.3.3. Discussion

As shown in this chapter, format of Big Data processing is dependent on the data type used. The most general classification includes batch and streaming data. Based on the data type we can specify which algorithm for efficient data processing can be used. This is the first phase of the data processing. Then we have to think what kind of information we expect as a result of data processing. The last step will be to use the processing method. Therefore, to conclude we can say that the decision of choosing the format of Big Data processing consists of answering three questions:

1) What type of data do we have?
2) Which format, tool and algorithm is the best to process the data?
3) What kind of information do we expect and which method is the best to do such analysis?

Each data source is different and may require a different method of data processing. We cannot say that one format of Big Data processing is the best, because it may only be suitable for one data source. Sometimes it is recommended to compare different methods/libraries/formats and decide whether performance, data space or data integration are the issues or not.

3.3.4. Literature

[1] http://apache.org [as of 20.12.2017].
[2] http://python.org [as of 20.12.2017].

3.4. Data-hub and Data-lake

3.4.1. Introduction

Sharing of multiple data sources is greatly facilitated when a single point of access is set up via which these sources are made available to others. In this way, all data needed by various users is available at a single location. As a result, there is no need to obtain access to a range of maps on the network or a number of networks. The data can be stored in their native (raw) form or may be available as structured, such as in a relational database, semi-structured, such as XML, CSV, JSON, or unstructured, such as emails, documents, data and blobs, in various formats. It could even contain binary data, such as images and video.

A data-hub and a data-lake are examples of access points for the distribution and sharing of large data collections. Some call these access points even virtual databases (Feldman, 2018). What a data-lake and data-hub have in common is the fact that all data is physically stored in it; all data has been transferred from its original location to the access point. The most important difference is that the data in a data-lake is *only* available in its original format (Stein and Morrison, 2014). The data is usually also not indexed, making a data-lake a location were data is dumped. A data-hub, in contrast, contains data that is checked, indexed and made available in a single standardized format. Some data-hubs even make data available in multiple formats (Chisholm, 2008).

3.4.2. Examples and methods

An easy way of creating a data-lake is storing all data in a Hadoop cluster or in a NoSQL database such as MongoDB. The “sandbox” environment of the UNECE is an example of an Hadoop based environment in which tools are available (UNECE, 2016). Storing data in the cloud is another way;

Amazon S3 and Azure Data Lake are examples of this. Downside of a data-lake is the absence of data maintenance. Since the primary purpose of a data-lake is data storage, each user has to find out the pros and cons of (subsets of) data on its own including any quality issues. The most often mentioned comment against using data-lakes is the fact that it can change into a 'data swamp' when it is not very actively used. The latter 'swamp' refers to a data storage facility that has become deteriorated and difficult to access because of the lack of control (Olavsrud, 2017). Such a data storage facility has little added value. As a result of this downside, alternatives for data-lakes have emerged in which various forms of 'cured' data are stored; this is why data-hub have been created.

In the ESSnet Big Data no WP has studied or mentioned a data lake as an option to store data. At Statistics Netherlands, however, a data lake is one of the options considered to store the raw data of various data channels. Most important benefit is that all data is available at a single location which can be used by many other processes. Major downside is the fact that for each foreseen use the data may need to be cured and combined. At the moment no definitive decision has been made.

A data-hub is a somewhat more advanced alternative for a data-lake. Again the data is made available in a centralized way but it has the additional advantage that the data are quality checked. The latter means, for instance, that the data is de-duplicated and standardized. Additional advantage of a data hub is that users are able to add value to the data, resulting in a considerable quality improvement specifically for the users within an organization. This aspect solves one of the big downsides of only sharing raw data. Because the data needs to be checked, curated and converted, this will take a considerable effort on the organization to do that. However, compared to maintaining all data in separate databases or in a single warehouse, it is expected to be somewhat more efficient. Apache Solr is an example of a storage facility with data-hub capabilities. The Italian statistical office uses such an environment.

In the first deliverable of WP3, in section 3.2, the way smart meter data is or is expected to be obtained is described for a number of countries (WP3 Del 3.1). Only Denmark and Estonia have access to smart meter data and both via a data-hub. Denmark uses the Danish data-hub for energy data. This data-hub is administered by EnergiNet and contains all data from Danish energy markets. Statistics Denmark connected with EnergiNet in 2015 to obtain access to this data. Downside of the hub is that information about the produced and consumed electricity of costumers is lacking; the data-hub only contains information on the electricity sold and bought (WP3 Del 3.2). Estonia also uses a data-hub in their country to obtain access to smart meter data. The Estonian Data hub has been initiated by Elering AS in 2012. Statistics Estonia discussed and obtained access in 2015. The hub contains all measurement data including agreements related to electricity transfer and consumption. Each year, the Estonian statistical office gets a complete copy of the data in the hub. The statistical offices of Norway and Sweden both mention that a hub containing smart meter data is expected to be set up in their country in the near future (Table 2 in WP3 Del 3.1). For all other countries either no plans have been developed to study smart meter data or there is no access to the data.

3.4.3. Discussion

Compared to other solutions of sharing data, both a data-lake and a data-hub are ways to provide access to large amounts of data. Advantage of a data-hub is that the quality of the data is checked and is available in a standardized format. Sometimes feedback of the users is included to improve the quality even more which greatly increases the use of the data by many analysts within an organization.

Organizing the data curation efforts in a data-hub takes more effort compared to merely sharing the data in its raw form via a data-lake. It is strongly suggested that NSI's try to influence what kind of data is stored in the hub, to assure all data relevant for official statistics production is available. However, even in a hub, combining various data sets is expected to remain challenging as it can be assumed that a considerable part of the data needs to be cured to enable this. This puts a considerable effort on each data analysis task and indicates the need for data access points with data of an even higher quality for official statistical purposes.

3.4.4. Literature

[1] Chisholm, M. (2008) Six Architectural Styles of Data Hubs. Located at: http://www.b-eye-network.com/view/8109

[2] Feldman, D. (2018) Data Lakes, Data Hubs, Federations: Which One Is Best? Blog available at: https://www.marklogic.com/blog/data-lakes-data-hubs-federation-one-best/

[3] Olavsrud, T. (2017) 3 keys to keeping your data lake from becoming a data swamp. Available at: https://www.cio.com/article/3199994/big-data/3-keys-to-keep-your-data-lake-from-becoming-a-data-swamp.html

[4] Stein, B., Morrison, A. (2014) The enterprise data lake: Better integration and deeper analytics. *Tech. Forecast* 1, pp. 1-9.

[5] UNECE (2016) Sandbox. Link: https://statswiki.unece.org/display/bigdata/Sandbox.

[6] WP3 Del 3.1 (2016) Smart meters: Report on Data access and Data handling. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_1.

[7] WP3 Del 3.2 (2016) Smart meters: Report on production of statistics: methodology. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_2.

3.5. Data source integration

3.5.1. Introduction

In statistical offices there is a need for an environment on which data sources, including Big Data, can be easily, accurately and rapidly integrated. This was already the case with administrative sources whose integration would enrich survey data for example. Big Data is, on the one hand only an additional data source, but on the other hand a different one which brings in more complexity in data processing and integration. So whenever we require more incorporation than what would be possible with a data lake or with a data hub, we have to resort to data integration.

This chapter focuses on integrating several data sources, among them Big Data, and the associated techniques that can be used based on the data type, the challenges felt in the pilots and possibilities available of data source integration with data residing on Relational Database Management System (RDBMS). For less structured environments as data lakes or data hubs please refer to Section 2.4.

3.5.2. Examples and methods

Nowadays to produce statistics NSIs must manage small and big, structured and unstructured data, sometimes batch and real-time streaming processing and even in some cases on-premises and cloud or other hybrid deployments.

In the following table we have a compilation of the data integration that is being performed in the pilots of the ESSnet on Big Data.

Table 5. Data Integration use cases in WPs

WP partner	Source description	Source Volume	Structured Unstructured	Source	Processing	Metadata Integration
WP1DE	Self-scraped	~ 60,000 p/ portal	Structured (xlsx)	Web	-	-
WP1DE	CEDEFOP scrap.	2,14 M.	Semi-Structured	Web	CDC	Machine learning
WP1DE	Administrative Data	~1 M	Semi-Structured	Federal Employment Agency	Delivery	Machine learning
WP1 SL	Scraped data	~13MB/week 64.000 rec/week	Semi structured	job portals	Matching, deduplication	
WP1 SL	Scraped data (enterprise site)	~200MB 5000 files	Unstructured (HTML)	Enterprise websites	Net address matching	Machine-learning
WP1 SL	Secondary (administrative) source	1MB/month 110.000 records/month	Structured	Employment Service of Slovenia	None	
WP1 UK	Scraped data (job vacancy counts per company)	~4million 7 sources	Semi structured (HTML)	Web	Matching, outlier detection	
WP1 UK	First 3rd party source (job vacancies)	~6GB (~42million records)	Structured (csv)	Provided by company	Aggregation (Pandas)	
WP1 UK	Second 3rd party source	~4million records	Structured (csv)	Provided by company	Aggregation (Pandas)	
WP2	Big Data	80k sites	Unstructured	Enterprises Web sites	Text mining	URLs of Enterprises
WP3	Electricity metering data – admin data	2TB in original format, 200GB ORC in Hadoop	Structured	Smart meter	Once a year full update	Linking by registry codes or address ID-s
WP4	AIS	huge	Structured	Ship	Other	Visualizing routes, calculating indicators
WP5 BE	Big data	395 billion records	Structured	Mobile phone net	analysis by SAS	Metadata very limited and unproblematic
WP5 FI	Admin data	medium	Structured	Registers	Standard	
WP5 FI	Survey data	small	Structured	Survey	Standard	
WP5 FI	Big data	huge	Structured	Telecom	N/A	
WP5 FR	Big Data	2-3 To	Well structured	Mobile phone CDR)	Offline, old research data	Integration only of aggregated data. Linked by geographic coordinates
WP5 NL	Signaling data	60GB/day 1.5 billion records	Structured	Mobile Network	Hadoop	Aggregated
WP5 NL	Municipal Personal Record Database	Admin, aggregated	Structured	Admin		Aggregated
WP6	Big Data	~30GB 3500 files	Semi-structured	Road sensor	Python, R	
WP6	Survey Data	Small	Structured	Turnover in industry	SAS, R	
WP6	Survey Data	Small	Structured	Economic sentiment indicator	Excel, R	
WP7	Administrative data	medium	Structured	Registers	ETL	The training fields selection criteria Data segmentation, classification, aggreg. Results assessment (quality of output)
WP7	Survey data	small-sized	Structured	Surveys	ETL	The training fields selection criteria Data segmentation, classification, aggreg. Results assessment (quality of output)
WP7	Big Data	huge	Unstructured	Satellite	Machine Learning	The training fields selection criteria Data segmentation, classification, aggreg. Results assessment (quality of output)
WP7	Big Data	huge	Unstructured	Web	Web scraping (flight movement)	Aggregation
WP7	Big Data	huge	Unstructured	Social Media	Machine Learning	Automatic classification of the source
WP7	Big Data	huge	Structured	Road sensors	Entropy Econometrics	Clustering, Estimating
WP7	Big Data	huge	Unstructured	Web	Web scraping Mach. Learn.	Automatic classification of the source

As it's visible in the table above in all cases the Big Data is just another source that has to be integrated with pre-existing data. However this integration can be made harder by the type of data (see Figure 1) and by the different processing path required by Big Data.

Figure 2. Big Data Processing Path

Although in the industry the steps of data discovery, data analysis and data processing are usually seen as separate steps in the Big Data processing path this is not the case in statistical offices. It's understandable that the consumers of the results from data discovery, i.e. the users of the data performing the analyses of the results are in most cases general information users and managers and only 5% of them have advanced statistical knowledge [1]. In Statistical offices the situation is reversed and probably only 5% of the demand will be covered by simply counting or adding values.

In fact this process of discovery, interpretation, and communication of meaningful patterns in data is what we perform as data analytics. Analytics relies on the simultaneous application of statistics, computer programming and operational research and as such the Statistical Big Data Processing Path introduced in Figure 3 will be used from now on as it depicts more accurately our situation.

Figure 3. Statistical Big Data Processing Path

The data on RDBMS follows a distinct path. If we focus on WP7 which has a very good example of combining Big Data, Administrative Sources and Survey data we see that while for processing Big Data they rely on Machine Learning for the other type of data they use Extraction, Transformation and Loading (ETL). It especially includes agricultural data processing of satellite images, in-situ surveys and administrative data according to the linkage process. Both in-situ survey and satellite images are used by machine learning algorithm to predict the crop type.
Before we introduce the methods for integrating the Big Data with other sources its worthwhile to take a look at the Relational Data processing path.

Figure 4. Relational Data Processing Path

As we can see the processing of the data from an Online Transaction Processing (OLTP) system suffering Extraction, Transformation and Loading (ETL), Extraction, Loading and Transformation (ELT)

or Change Data Capture (CDC) happens before the storage on the Relational Database Management System (RDBMS) and the analytics only take place at the very end of the process [2].
This gives us a good connection point for integration. If we develop a data bus using metadata and semantic technologies, which will create a data integration environment for data exploration and processing.

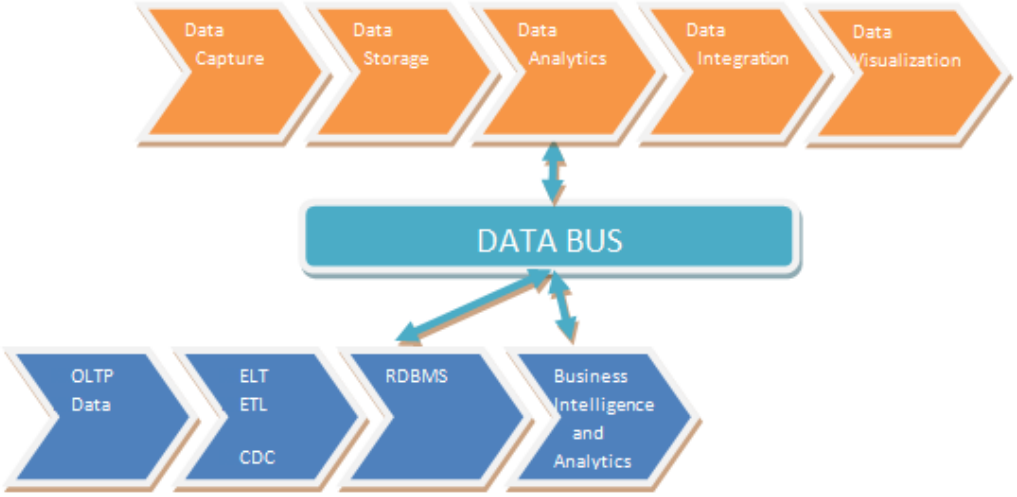


Figure 5. Data Integration using a Data Bus

With this strategy it will be possible to maintain an heterogeneous physical architecture. The main complexity will be on the data bus architecture and on the metadata. Although data integration can become a performance bottleneck we will be able to have a scalable design both for RDBMS and Big Data processing [3].

Another possibility for data integration is to use a connector to exchange data between the two platforms. In this case the connection would be done at the storage level on both platforms.

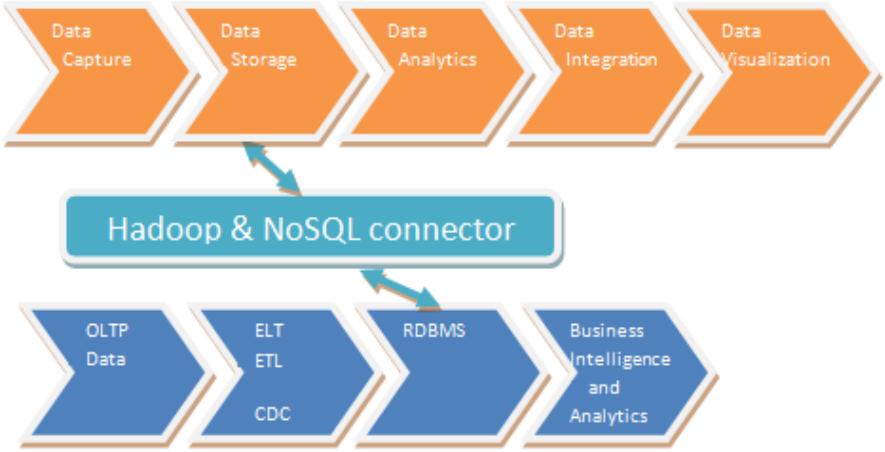


Figure 6. Data Integration using a Connector

The weakness of this approach will be the performance of the Big Data connector. We maintain the heterogeneous physical architecture with the same advantages as in the data bus relative to the scalability. The metadata architecture and management is not as critical but the queries themselves can become complex.

Most RDBMS vendors [4] are not only offering Hadoop and NoSQL connectors but also appliances to build an integration layer [5] between the RDBMS and Hadoop. The appliance would build the bridge in a similar position as the connector, i.e. at the storage level but it can be more costly in terms of the customized configuration maintenance that will be required.

Another option would be to introduce a new layer for semantic data integration with data virtualization built on the RDBMS.

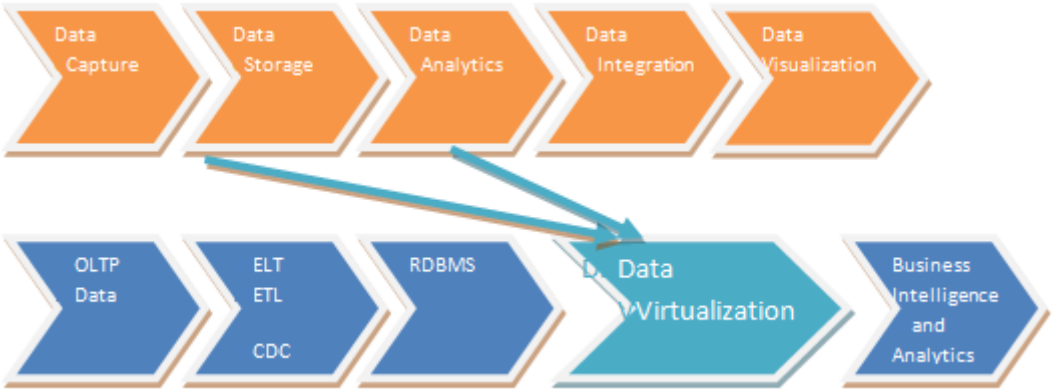


Figure 7. Data integration using Data Virtualization

This approach would perform the data integration from the Big Data side both at data storage and data analysis level while introducing a new layer on top of the RDBMS to perform the semantic data integration.

Here we maintain scalability and the workload is optimized although the maintenance of the new integration layer will require heavy maintenance.

3.5.3. Discussion

When we are requiring data integration of a traditional source (Statistical survey or administrative data) with a Big Data Source we have on one hand a consolidated model and on the other a data driven model, which is unknown before the data Analysis or at least until the data discovery. In the past data integration techniques have been focused on ETL, ELT, CDC, and EAI types of architecture but to suit the size and processing complexity demands of Big Data, including the formats of data that need to be processed means to adopt a data-driven integration perspective.

Another problem is that the current infrastructure installed where data integration is being carried out until now will no longer support the needs of the Big data on the same platform. The structure for data integration has to be highly flexible and scalable from the architecture perspective.

We have seen several possibilities to perform data integration. Many more exist both in terms of platforms and phases to perform the integration. There is not a magical and unique answer to this problem but the data type should be taking into account and the flexibility and scalability preserved.

3.5.4. Literature

[1] Handbook on the Statistical Data Wareouse on the Cros Portal - https://ec.europa.eu/eurostat/cros/content/s-dwh-design-manual_en last visited 20/2/2018

[2] Dijcks J. P. and Gubar M. (2014): Integrating SQL and Hadoop. Business Intelligence Journal, Vol. 19, No. 2.

[3] Centre of Excellence on Data Warehouse (2017): Statistical Data Warehouse Design Manual – Architecture Chapter. EUROSTAT available on https://ec.europa.eu/eurostat/cros/content/architecture_en last visited 20/2/2018

[4] Steiner J. (2011): Managing Big Data by Using Hadoop and Oracle Exadata. Oracle White Paper, 2011

[5] Beyer M. A., Thoo E., Selvage M. Y. and Zaidi E. (2017): Magic Quadrant for Data Integration Tools. Gartner report, August version, ID Number: G00314940

3.6. Choosing the right infrastructure

3.6.1. Introduction

A number of Big Data oriented infrastructures are available. Choosing the right one for the job at hand is key to assuring optimal use is made of the resources and time available.

Big data endeavours require, to bigger or lesser extent, to opt for infrastructures that can guarantee:

- *Linear scalability*. In terms of storage, memory, and processor.
- *High throughput*. Big Data's velocity mandates that data be ingested and processed at high speeds. That means to be extremely fast across input/output (I/O), processing, and storage.
- *Fault tolerance*. Big Data because of its inherent complexity needs a fault-tolerant architecture. Any one portion of the processing architecture should be able to take over and resume processing from the point of failure in any other part of the system.
- *Auto recovery*. The processing architecture should be self-managing and recover from failure without manual intervention.
- *Programing language interfaces*. Big Data can be processed for multiple business scenarios which makes it difficult to use any COTS(commercial off-the-shelf) software and needs custom coding and development.
- *High degree of parallelism*. By processing data in parallel, we can distribute the load across multiple machines, each having its own copy of the same data, but processing a different program.
- *Distributed data processing.* Since Big Data processing happens on a file-based architecture, to achieve extreme scalability, the underlying platform must be able to process distributed data. This is an overlapping requirement with parallel processing, but differs in the fact that parallelism can exist within multiple layers of the architecture stack.

All these requirements translate into building blocks [1] that form our Platform. The resulting platform will then be able to serve the Big Data Life Cycle and thus provide landing zones for the Data, forms of ingestion, ways of processing, discovery enablers and to support outputs from all this processes as analytics providers, database integration and reporting.

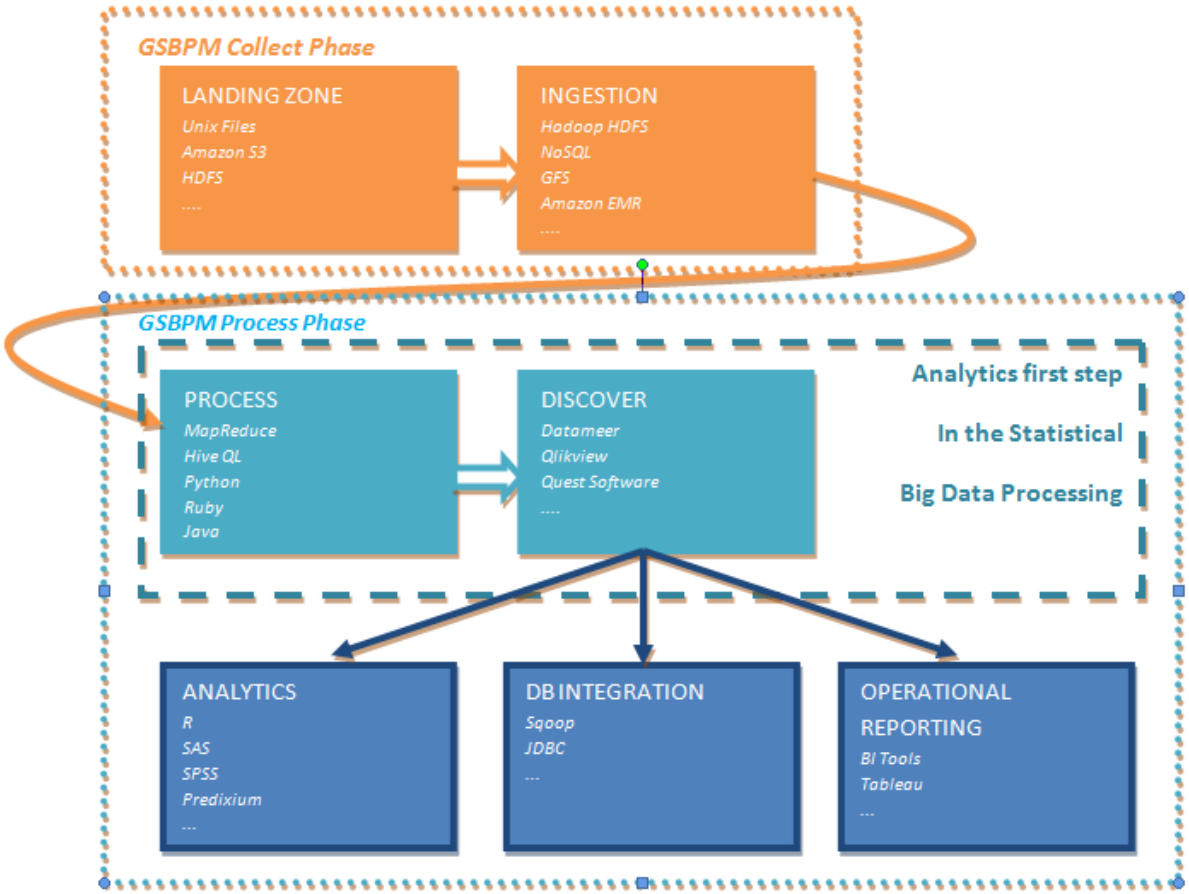


Figure 8. Conceptual Big Data Platform

In the figure above we introduce a rough sketch of what a Big Data Platform could be [2], with some examples of technologies that can be used in every block. Naturally different Big Data projects have distinct requirements. Not always the same building blocks will be required while sometimes they will be used in cycles, going from process to discovery and back to process again, for example. For Big Data Processing life cycle please refer to chapter **Fout! Verwijzingsbron niet gevonden.**. The goal of the present chapter is to present the possible big data infrastructures and provide heuristics for infrastructure choice to specific problems.

3.6.2. Examples and methods

When dealing with Big Data Statistical Offices have different requirements than the general industry not only in terms of data integration but also throughout the processing of Big Data, for example the ingestion may not need to use streaming for ingestion due to lower dissemination frequency, days instead of seconds. In terms of volume the difference between projects can also be huge so here also the processing capability and storage can vary a lot. To provide us with a more comprehensive guide an inventory of technologies used across the Big Data Platform was compiled, for all pilots in the ESSnet on Big Data. The list, following the Conceptual Big Data platform schema introduced in the last section, is presented in the following table.

Table 6. Comprehensive list of what is being used across the ESSnet on Big Data

Big Data Phase	Building Blocks
Landing Zone	Linux, Windows, HDFS and HDFS over HUE
Ingestion	Selenium – scripting language CSV, JSON, MongoDB, Google Cloud, Hadoop, NoSQL, HDFS – data storage, Apache Solr, Elastic Search
Process	Java, Python (Orange, Pandas), SAS, R – programming languages Hive, HDFS – data storage Spark – processing engine
Discovery	SAS, Orange, Python, Sklearn, Spark, Kibana, R
Analytics	RStudio, Python, SAS, Excel, Orange, Sklearn, Jupyter, Spark, R, QGIS
DB Integration	MySQL, MariaDB, SAS, Orange, GAMS, Spark
Operational Reporting	RStudio, Shiny, Apache POI, SAS, Orange, R, Excel

We see here a humongous paraphernalia of building blocks whose variety comes not only from the natural diversity between NSIs but also from the challenges presented by the pilots themselves. Some of the requirements identified in section **Fout! Verwijzingsbron niet gevonden.** can be better understood through this table. Particularly in the Process, Discovery and Analytics it is clear that COTS software doesn't cover the needs. This was also refered directly in almost every project, mentioning the large number of specific libraries used, be it in R or Python.

In different degrees, depending on the specific project, other requirements map to other solution as HDFS over HUE in the landing zone or the use of Google cloud for ingestion. Let's try to organize these requirements and translate them into infrastructure options.

There are several forms to increase our capacity of storage or processing [3]. Since we are considering processing of large amounts of data here, there are in principal two ways to scale the processing of data: *horizontal* or *vertical*. In both cases more data can be processed within the same or within less time.

In *horizontal* scaling the workload is distributed across many servers. It is also known as ´scale out´, where multiple independent machines are added together in order to improve the processing capability. The hardware entities are connected in such a manner that they function as a single logical unit. It can practically scale infinitely, although there are some limits imposed by software or other attributes of an environment's infrastructure. Compared to vertical scaling, this is a less expensive option which can be gradually achieved. Downside is that the data has to be divided and processed over several machines which requires more complex data handling.

In *vertical* scaling more processors, more memory and faster hardware, are typically installed within a single machine. It is also known as ´scale up´ and it usually involves a single instance of an operating system. It helps in upgrading the capacity of the existing server. Compared to horizontal scaling this approach is more expensive and there is a limit to how far one can upscale. Big advantage is that all data is processed within a single machine which makes this easier to control.

In each scaling direction a number of solutions are available. Within the context of WP8, especially the ability of each solution to adapt to increased data processing demands is an important consideration

as is the ability to process data in a secure protected environment. Apart from that some other infrastructure considerations are discussed.

Horizontal scaling

In the horizontal scaling direction a number of options are available. The one which will be briefly discussed here are: Peer-to-peer networks, Hadoop/MapReduce and Spark.

Peer-to-peer networks can involve millions of machines all connected in a network. It is a decentralized and distributed network architecture where the nodes in the networks (known as peers) serve as well as consume resources. It is one of the oldest distributed computing platforms in existence. This setup has been used for several of the most well-known filesharing networks on the internet, such as Napster and Bittorrent. Major downside of these networks is the overhead caused by the communication between the nodes. A peer-to-peer network setup is also used for Folding@home, a project that studies protein folding by making use of the massive amount of computational power provided by volunteers. Here, however, the focus is more on computing at each node and less to network communication. Out scaling of peer-to-peer networks is easy but because the nodes can reside anywhere on the internet secure data sharing is not really an option.

Hadoop/MapReduce is a framework for storing and processing large amounts of data using clusters of (usually) commodity hardware. It is designed to scale out to thousands of nodes and is highly fault tolerant. The latter is done by making sure all parts of the data are distributed over multiple nodes. When one of the nodes crashes, the data that resided on it is still available on several of the other nodes. The programming model used on Hadoop is MapReduce which was originally proposed by Google employees . It can be applied to Petabytes of data. All processing steps are broken down into two procedures; a map and a reduce step. Mappers perform filtering and sorting while Reducers perform a summary operation. Major downside of the MapReduce/Hadoop is its inefficiency in running iterative algorithms as it is not designed for it. During each step data is read from disk and results are written to disk, making disk access the major bottleneck. Some attemtos were made to deal with these issues.

*Spark i*s designed to overcome the major drawback of Hadoop/MapReduce. It still uses the MapReduce paradigm but has the ability to perform in-memory computations. When the data can completely fit into memory, Spark can be 100x faster compared to Hadoop and when it resides on disk it can be up to 10x faster. Henceforth, Spark is gradually taking over Hadoop/MapReduce systems. The ability to deal with large amounts of streaming data is a very attractive feature of Spark.

Vertical scaling

Vertical scaling options are High Performance Computing Clusters, Multicore processors, Graphics Processing Units and Field Programmable Gate Arrays. These will all be briefly introduced below.

High Performance Computing Clusters are machines with thousands of cores. They are composed of high end hardware of which, depending on the user requirements, the amount of disk space, memory and overall setup may vary. The way to distribute the tasks and data over the various cores is usually via the Message Parsing Interface (MPI). Downside of MPI is its lack of capability to handle faults but because of the high quality of the components used in High Performance Clusters this is usually not a

major drawback. When dealing with faults is required, MapReduce can be used as an alternative to MPI.

Multicore refers to processors, i.e. Central Processing Units (CPU's), with large amounts of cores. Such a machine can have up to dozens of cores which shared memory and often have a single large disk. More and more of the processors in commodity hardware have a considerable number of physical cores. Sometimes this amount is doubled by what is known as multithreading; a process that improves parallelization by assigning two virtual cores to each physical core. As a result, a considerable gain can be achieved in processing large amounts of data by distributing the data over all cores available. Drawback of this approach is the limitation to the absolute number of cores available on CPU's and the limitation to the speed by which data can be accessed. If all data fits into memory this is less of a problem -although CPU's can process data faster than memory access speed- but for amounts of data that exceed the systems memory, disk access becomes a huge bottleneck. Caching cannot solve these issues for a CPU.

Graphics Processing Units (GPU's) are specialized hardware designed to accelerate the creation of images in a frame buffer intended for display output. They are predominantly used for gaming purposes. Due to their massively parallel architecture, recent developments in GPU's hardware and related programming frameworks have given rise to General-Purpose computing on GPU's. Since many GPU's have 2500+ cores and fast memory available, they can be used to rapidly perform massive amounts of calculations. Major downsides of GPU's are the lack of communication between various parallel processes and the limited ability to process huge amounts of data. The latter is the result of the limited amount of memory available on GPU's which makes the speed of disk access the major bottleneck.

Field Programmable Gate Arrays (FPGA) are highly specialized hardware units which are custom-built for specific applications. They can be highly optimized for speed and can be orders of magnitude faster compared to other platforms for certain applications. Programming is done with a hardware descriptive language which requires detailed knowledge of the hardware used. Because of this, development costs are typically much higher compared to other platforms. Only for very specific uses, such as near real time processing of huge amounts of data collected by large scale astronomic instruments, this approach might be beneficial. This option is sometimes referred to as scale-deep.

Other considerations

The ability to smoothly and continually add compute, memory, networking, and storage resources to a given node or set of nodes that make up a larger computing environment can also be achieved using the Public clouds like Amazon AWS, Microsoft Azure and Google Cloud. This form of scaling is called **hyperscale.**

Other ways to reach scalability are using algorithms that embrace parallelism such as NoSQL-like schema-less storage, and data sharding. Data sharding and the ability to storage schema less data in dynamic columns is present also in the refered MongoDB and MariaDB, both **NoSQL like solutions**.

3.6.3. Discussion

Both **scale-up** and **scale-out** approaches [4] are valid means of adding computing resources to a **data** center environment, and they are not mutually exclusive. **Scale-deep** [5] is only required in very specific and niche problems.

Scale-up or vertical scaling was the hardware answer to getting data at a rate, or Velocity, beyond what former approaches could handle. It can be the best option to perform real-time processing. Implementation is not difficult and administrative costs will be reduced as may be the licensing costs, although more recent license policies take into account the number of cores used. It may have reduced software development costs and simplified debugging. Having more cores can also offer a more consistent performance as the distribution of loading/processing may not be constant but subject to spikes.

Scale-out or horizontal scaling was the hardware solution to getting data with a Volume beyond what the older approaches could handle. It allows us to use smaller systems, resulting in a cheaper option. It's easy to upgrade or scale-out further. It's resilient because to the multiple systems. Due to the same reason it's easier to run fault tolerance. It supports a high degree of parallelism being a good match for algorithms that embrace parallelism such as MapReduce or Spark, NoSQL-like schema-less storage, and data sharding.

The technologies used and adopted by the pilots in the ESSnet on Big Data confirm horizontal scaling as being preferable and the best options when the real-time processing is not at stake. This finding is in agreement with the findings of Sing and Reddy (2014). They used a score from 1 to 5 stars for the horizontal and vertical scaling options discussed above. 5 is the best rating. Their table is reproduced below.

Scaling type	Platforms (Communication Scheme)	System/Platform			Application/Algorithm		
		Scalability	Data I/O performance	Fault tolerance	Real-time processing	Data size supported	Iterative task support
Horizontal scaling	Peer-to-Peer (TCP/IP)	★★★★★	★	★	★	★★★★★	★★
	Virtual clusters (MapRedce/MPI)	★★★★★	★★	★★★★★	★★	★★★★	★★
	Virtual clusters (Spark)	★★★★★	★★★	★★★★★	★★	★★★★	★★★
Vertical scaling	HPC clusters (MPI/Mapreduce)	★★★	★★★★	★★★★	★★★	★★★★	★★★★
	Multicore (Multithreading)	★★	★★★★	★★★★	★★★	★★	★★★★
	GPU (CUDA)	★★	★★★★★	★★★★	★★★★★	★★	★★★★
	FPGA (HDL)	★	★★★★★	★★★★	★★★★★	★★	★★★★

This table reveals the poor scalability of vertical scaling in contrast to horizontal scaling. The reverse is observed for the support of large data sizes. For Big Data processing and analysis clearly horizontally scalable systems are to be preferred. Since Peer-to-peer networks have a poor I/O performance and fault tolerance, Hadoop/MapReduce and Spark are the best options. Of these two, Spark has the best I/O performance and iterative task support. Downside of all horizontally scaled systems is there poor real-time processing capabilities. For such tasks, a vertically scaled system is a better option. GPU's

excel in this area if the data size remains small. For real-time processing of large amounts of data, High Performance Clusters are the best alternative. The simplest way to speed up the processing and analysis of large data is distributing the data on all available cores on the machine in use.

3.6.4. Literature

[1] Kune R. ET AL.(2016): The anatomy of big data computing. Published online 9 October 2015 in Wiley Online Library DOI: 10.1002/spe.2374

[2] Lehman D., Fekete D. and Vossen G.: Technology Selection for Big Data and Analytical Applications. European Research Center for Information Systems (ERCIS) Working Paper No. 27

[3] Singh D. and Reddy C.K. (2015): A Survey on Platforms for Big Data Analytics. Journal of Big Data, Springer 2015

[4] Phule R. and Ingle M. (2013): A Survey on Scalable Big Data Analytics Platform. International Journal of Science and Research (IJSR) ISSN (Online): 2319-7064

[5] Kepner J (2017).: Enabling Scale-Up, Scale-Out, and Scale-Deep for Big Data. MIT Lincoln Laboratory. Supercomputing center

3.7. List of secure and tested API's

3.7.1. Introduction

Collecting information from websites is a process that can be implemented with traditional web scraping, manually or automatically. Usually it means that the person who scraps the website must be familiar with the construction of HTML (Hypertext Markup Language) website, its tags and CSS (Cascade Style Sheet) classes, to develop a robot that allow transforming web based semi-structured information into the data set. Because the website owners can block a robot when massive web scraping is running or they could limit the access for robots with Captcha codes, it is highly recommended to discover if any API's are provided by the website owners.

An application programming interface (API) is a set of subroutine definitions, protocols, and tools for building application software. It is important to know which API´s are available for Big Data and which of them are secure, tested and allowed to be used. Using the API allows to omit any legal issues regarding web scraping. If the data owner provides an API interface, the rules of accessing the data are also described. For instance, with Twitter API you have limits in the number of requests. Most of the issues are listed in 3.7.2. Some of the API's are not available for free and different pricing plans allows to access more detailed or historical data. For example, flightaware.com, that allows to access historical data on flights, has five different pricing plans available [1].

The goal of this chapter is to present the list of API's used for statistical purposes in different projects. It includes the characteristics of each API with its basic functionality and possible use in different statistical domains.

3.7.2. Examples and methods

From official statistics point of view, we need to examine the API's that were used with success to collect information for statistical purposes. The list of them is presented in

Table 7.

Table 7. Brief overview of API's

No.	Name of the API with hyperlink	Basic functionality	Restrictions	Potential domains (WP number)	Remarks
1	Twitter API	Scrap the tweets by keywords, hashtags, users; streaming scrapping	25 to 900 requests/15 minutes; access only to public profiles	Population, Social Statistics, Tourism (WP2, WP7)	Account and API code needed
2	Facebook Graph API	Collect information from public profiles, also very specific such as photos metatags	Mostly present information, typical no more than dozens of requests	Population (WP7)	Account and API code needed
3	Google Maps API	Looking for any kind of objects (e.g., hotels), verification of addresses, monitoring the traffic on specific roads	Free up to 2.5 thous. requests per day. $0.50 USD / 1 thous. additional requests, up to 100 thous. daily, if billing is enabled.	Tourism (WP7)	Google account and API code needed
4	Google Custom Search API	Can be used to search through one website, with modifications it will search for a keywords in the whole Internet; can be used to find a URL of the specific enterprise	JSON/Atom Custom Search API provides 100 search queries per day for free. Additional requests cost $5 per 1000 queries, up to 10k queries per day.	Business (WP2)	Google account and API code needed
5	Bing API	Finding specific URL of the enterprise	7 queries per second (QPS) per IP address	Business (WP2)	AppID needed
6	Guardian API	Collect news articles and comments from Guardian website	Free for non-commercial use. Up to 12 calls per second, Up to 5,000 calls per day, Access to article text, Access to over 1,900,000 pieces of content.	Population, Social Statistics (WP7)	Registered account needed
7	Copernicus Open Access Hub	Access to Sentinel-1 and Sentinel-2 repositories	Free for registered users	Agriculture (WP7)	Registered account needed

The list shown in

Table 7. includes basic set of API's already used for statistical purposes. All of them are constructed to handle requests prepared in a specific format, e.g.,

```
http://api.bing.net/xml.aspx?Appid=<AppID>&query=bigdata&sources=web
```

is a formatted request for Bing API to get the results in JSON (JavaScript Object Notation) format on searching through the `web` for `bigdata` term. The results of the requests, depending on the API, may be formatted to JSON or XML (Extensible Markup Language) files.

Therefore, the listed API's are not dependent on the programming language. Although most of the API's has a substitutes in libraries, such as Tweepy is a Python library to access the API directly from this language, usually recommended option is to used universal libraries. Our experience shows that name of the classes and methods in different libraries may change, which make it difficult to maintain a software using them. Using the API libraries also makes it necessary to register and generate the API key to scrap the data. The best known API in Big Data projects for statistical purposes is a Twitter API. For this social media, several different libraries exist for different languages. One of them is Tweepy that allows access via API without formulating the requests text. Different parameters allow accessing the social media channel and store the results in Python dictionaries.

3.7.3. Discussion

Using API's allows accessing the website or any datasets in more stable way than using traditional web scraping. For example, the structure of the website may be changed very frequently, resulting in changing CSS classes, which makes the software written to scrap the data very instable. Therefore, recommended solution is to find an API associated with the website that was designated to scrap. This is the major strength in using API comparing to scrap the data in traditional way.

On the other hand, API's have many weaknesses. They may also be very instable and continuing maintenance is important. One of the examples is a Google Search Engine API that was deprecated and changed into Google Custom Search API. It resulted in the necessity of changing the software source code to access a new API's for the same purposes but working different way.

As mentioned in the previous part, the recommended solution is to use API's instead of traditional web scraping by collecting the data directly from websites. However using API does not allow us to treat the software as final version, as API's are a living interfaces and may change its structure. Also we cannot be sure that API's will be supported by data owners all the time. In various situations, the development may be stopped or in the specific situation the pricing plans may change resulted in ceasing free access to the data source.

3.7.4. Literature

[1] http://flightaware.com/commercial/flightxml/pricing_class.rvt, accessed 9th of November 2017

[2] https://developer.twitter.com/en/docs, accessed 9th of November 2017

[3] https://developers.facebook.com/docs/graph-api, accessed 9th of November 2017

[4] https://developers.google.com/maps/, accessed 9th of November 2017

[5] https://www.bing.com/developers/s/APIBasics.html, accessed 9th of November 2017

[6] http://open-platform.theguardian.com/access/, accessed 9th of November 2017

[7] https://scihub.copernicus.eu/twiki/do/view/SciHubWebPortal/APIHubDescription, accessed 9th of November 2017

3.8. Shared libraries and documented standards

3.8.1. Introduction

Sharing code, libraries and documentation stimulates the exchange of knowledge and experience between partners. Setting up a GitHub repository or alternative ones would enable this.

Although Big Data is very often related to technologies such as Apache Hadoop or Spark, most of the Big Data work is done in programming languages such as Python, R, Java or PHP. The variety of programming languages and tools used, makes a necessity of creating the set of shared libraries and documented standards, that can be easily used by other users. In other words, it will allow executing the software by other NSI's without problems regarding software misconfiguration.

Common repositories provide many benefits to users. Firstly, there is a possibility of the version control. It means that every change in the source code is saved with history that can also have a description. This allows going back to any of the previous version, e.g., if the software is not consistent and stable after specific change in the source code. The second benefit is that software can be shared all the time with the public or private (authorized) users. Any change may be monitored and tested by them. Also very important in terms of software development is a possibility to discuss changes and give feedback. Finally, the repository has usually a common structure for documentation.

3.8.2. Examples and methods

Growing market of software development resulted in numerous repositories. Their main function is to share the software and provide version control with revision numbers. The difference is usually in additional features offered by the repository. Advanced repositories developed by commercial companies are usually not for free. However it is very common that light version, with limited functionality is offered for free to encourage persons to use a specific repository. In Table 8. we put the list of selected source code repositories that would enable to achieve the goal of sharing libraries and software.

Table 8. Main features of selected source code repositories

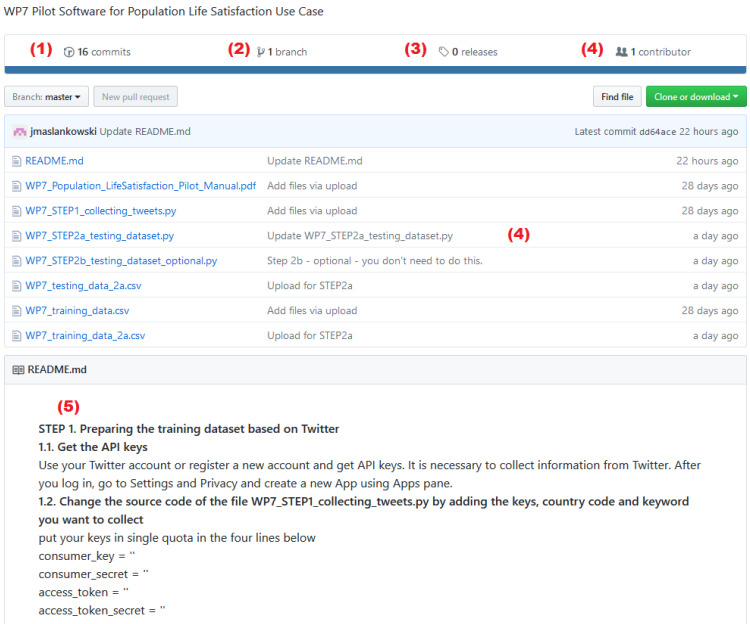
No.	Name	Link	Main features
1	GitHub	http://github.com	Most popular, free access, branches, etc.
2	Google Cloud Source Repositories	https://cloud.google.com/source-repositories	With connection to GitHub, Bitbucket or any other repositories on Google infrastructure, additional features include debugging.
3	Bitbucket	https://bitbucket.org	Can be integrated with Jira, up to 5 users per project for free.
4	SourceForge	http://sourceforge.net	Very common for software release, including project tracking and discussions.
5	GitLab	http://gitlab.com	Integrated wiki and projects websites.
6	Apache Allura	https://allura.apache.org	With the support for control version languages like Git, Hg and Subversion (SVN), internal wiki pages, searchable artifacts.
7	AWS CodeCommit	https://aws.amazon.com/codecommit	Mostly for AWS users, provide access to private Git repositories in a secure way.
8	GitKraken	https://www.gitkraken.com	Free version for up to 20 users, special features include visualization tools of the project progress.

The list presented in the table above shows the main repositories that can be used for free with some limitations listed in the main features column. As it can be seen, some of the repositories are dedicated for specific users, e.g., AWS cloud users, Jira or SVN users. Therefore, the decision of the use of the

specific repository will be connected with the tools used for software development. It is a reliable decision to use AWS integrated tools when working with AWS environment. However, in this document we will concentrate mostly on the most popular repository which is GitHub.

GitHub is structured in a specific way, where README file is the first file user can see when looking into the repository, like presented in Figure 9.

Figure 9. Typical structure of the project in GitHub repository



In the figure above, five different sections were indicated. Under the title of the repository there are four numeric data – about number of commits (1), branches (2), releases (3) and contributors (4). This information allows monitoring changes in the repository. The main section is indicated with the number (4). It is the list of files in the repository that can be cloned. The most important file for the first time users of the repository is a README.md file. It is the metadata of the project. The content of the file is written with HTML tags and displayed in section (5). This file should contain basic metadata on how to use or at least how to start to work with the repository.

The basic feature of the GitHub is the possibility of creating the clone of the source code. It is possible to install on computer the GitHub tool that allows to copy the remote GitHub repository into local machine with the same structure as the original repository. Then, it is possible to execute the software or modify it. For example a command:

```
git clone https://github.com/user/repository-name
```

will clone the repository of the specific user. The results of the clone of the repository is presented in Figure 10.

Figure 10. An example of GitHub clone process

```
jacekm@smtbigd00:~/temp$ git clone https://github.com/jmaslankowski/WP7-Populati
on-Life-Satisfaction          (1)
Cloning into 'WP7-Population-Life-Satisfaction'...
remote: Counting objects: 50, done.
remote: Compressing objects: 100% (47/47), done.
remote: Total 50 (delta 23), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (50/50), done.
Checking connectivity... done.
jacekm@smtbigd00:~/temp$ ls -l        (2)
total 4
drwxr-xr-x 3 jacekm spece 4096 lis 24 12:45 WP7-Population-Life-Satisfaction
jacekm@smtbigd00:~/temp$ cd WP7-Population-Life-Satisfaction/
jacekm@smtbigd00:~/temp/WP7-Population-Life-Satisfaction$ ls
README.md
WP7_Population_LifeSatisfaction_Pilot_Manual.pdf
WP7_STEP1_collecting_tweets.py
WP7_STEP2a_testing_dataset.py                                   (3)
WP7_STEP2b_testing_dataset_optional.py
WP7_testing_data_2a.csv
WP7_training_data_2a.csv
WP7_training_data.csv
jacekm@smtbigd00:~/temp/WP7-Population-Life-Satisfaction$
```

Three parts indicated in the figure above are the clone command (1), the result of creating the clone – a new directory with the project name appeared (2) and the content of the directory (3) which is the same as presented in Figure 9. The next step for the user is just to execute the software or use the cloned libraries.

In Table 9. the list of the well-known repositories dedicated for official statistics was presented.

Table 9. Popular GitHub repositories for official statistics

No.	Name	Link	Main features
1	Awesome Official Statistics software	https://github.com/SNStatComp/awesome-official-statistics-software	The list of useful statistical software with links to other GitHub repositories, by CBS NL
2	ONS (Office for National Statistics) UK Big Data team	https://github.com/ONSBigData	Various software developed by ONS UK Big Data Team
3	ONS (Office for National Statistics) UK Data Science Campus	https://github.com/datasciencecampus	Various software developed by ONS Data Science Campus Team
4	ESTP Big Data course software	https://github.com/SNStatComp/ESTPBD	Various software developed for the ESTP Big Data training courses

The list of the repositories presented in the table above may change over time. Therefore, it is recommended to watch the repositories from registered GitHub account.

3.8.3. Discussion

The benefits from sharing the libraries and software on repositories with versioning are strong visible especially when working in a group on one Big Data project. It helps to manage the revisions of the software produced, change the stages of the software development and inform the numerous users about changes or new releases of the software.

On the other side, programmers may be discouraged using the repositories when the project is not complex and only one person is developing the software. Alternative way of versioning is just to save different files with manual versioning. The reason for doing this is also to keep the software safe in one location. Although the repositories may be private and restricted from accessing them by other users, some users may not trust in the private policy.

To conclude, we can say that it is highly recommended to create the software with the support of repositories with version control. We recommend to share Big Data libraries and software created by

NSI's. It may result in the increased use of Big Data among official statistics users. As the consequence, the quality of the software will increase because of the wide group of users testing the software and giving feedback. Good practices of having public repositories were shown in Table 9.

3.8.4. Literature

[1] http://github.com, accessed 20th of November 2017

[2] https://cloud.google.com/source-repositories, accessed 20th of November 201

[3] https://bitbucket.org, accessed 20th of November 2017

[4] http://sourceforge.net, accessed 20th of November 2017

[5] http://gitlab.com, accessed 20th of November 2017

[6] https://allura.apache.org, accessed 20th of November 2017

[7] https://aws.amazon.com/codecommit, accessed 20th of November 2017

[8] https://www.gitkraken.com, accessed 20th of November 2017

[9] https://github.com/SNStatComp/awesome-official-statistics-software, accessed 20th of November 2017

[10] https://github.com/ONSBigData, accessed 20th of November 2017

[11] https://github.com/datasciencecampus, accessed 28th of February 2018

[12] https://github.com/SNStatComp/ESTPBD, accessed 28th of February 2018

3.9. Speed of algorithms

3.9.1. Introduction

It is important in this section to make clear from the start what is exactly considered an algorithm and what is considered a method. This is important because sometimes these words are used interchangeably which is not correct. Strictly speaking, an algorithm is considered a means to a method's end. In other words, an algorithm is the implementation of a method; usually in computer code. As a result, the following definitions are used:

> An *algorithm* is a set of instructions designed to perform a specific task. In computer programming, algorithms are usually composed of functions that are executed in a step-by-step fashion with the aim to terminate at some point.
>
> A *method* is a particular procedure to accomplish something in accordance with a specific plan. It can also be described as a systematic procedure to -in an orderly fashion- accomplish a task. An algorithm is a way to lay down such a procedure.

Because an algorithm is an implementation of a method, some of the choices made during the implementation affect its properties. The most important property considered in this section is the speed of the algorithm which is the amount of time needed to complete its task.

A number of factors affect the speed of an algorithm. One of the most important, but not the only one, is the exact way in which a method is implemented. How well this is done is commonly indicated by the general term 'algorithm efficiency' (Wikipedia, 2018a). In the context of this section, an algorithm that is maximally efficient consumes the least amount of time to fully complete its task. From a theoretical point of view, certainly when processing large data sets, the complexity of the algorithm is a very important contributor to the overall time needed to process data. In the field of computer science, this complexity is indicated by the so-called Big *O* notation (Bachmann, 1894). It expresses the time, as indicated by the number of operations, needed for an algorithm to complete its task as a function of the size of the input data (*n*). Various algorithms behave different when the amount of data they process increases. For algorithms the complexity notations listed in table 10 can be discerned; they are shown from fast to slow.

Table 10. An overview of the various algorithmic complexity notations discerned

Name	Notation	Examples
Constant	$O(1)$	Finding a minimum value in an array
Logarithmic	$O(\log n)$	Finding an item in a sorted array with binary search
Linear	$O(n)$	Finding an item in an unsorted list or malformed tree
Loglinear	$O(n \log n)$	Performing a Fast Fourier Transform, heap sort or merge sort
Quadratic	$O(n^2)$	Multiplying to n-digit numbers, bubble sort or insertion sort
Exponential	$O(c^n)$, $c > 1$	Determining if two logical statements are equivalent with brute force search
Factorial	$O(n!)$	Solving a traveling salesman problem with brute force search

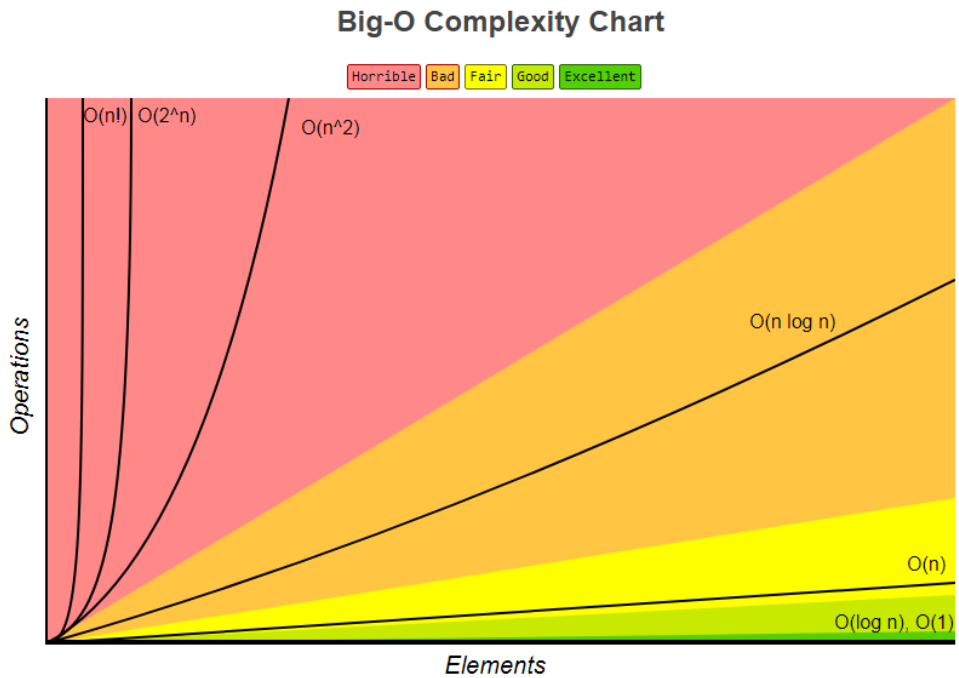


Figure 11. Big O complexity chart of algorithms. The number of operations are shown versus the number of elements (size n) for a number of complexity functions. (from http://bigocheatsheet.com/)

3.9.2. Examples and methods

Considerable decreases in the time needed to perform a particular task can be achieved by applying a less complex approach. For instance, changing from an algorithm with a quadratic complexity to one

with a linear complexity reduces the time needed to complete the task by the square root of *n*. However, not for every task an algorithm of a lesser complexity can be used. In such cases there are a number of other alternatives to can be considered. The most often mentioned are: i) using an 'approximate' approach (Almudevar, 2014) or ii) performing the task in parallel (Pacheco, 2011). Both approaches can be combined off course.

i) When an approximate approach is used, one decides not to go for the optimal, i.e. best, solution. This is especially useful when a lot of considerations need to be tested and/or when it is uncertain if an optimal approach exists or can be found within a reasonable amount of time. For some tasks this is the only way to obtain an answer during the life of the scientist.

ii) When implementing methods in parallel, the task is distributed over multiple devices. These can be multiple cores on the same processor, multiple processors on the same machine and/or on multiple machines. Each of these devices execute a part of the overall task and the results are combined at the end to get the correct answer. Parallelization can speed up tasks considerably but because of the distributed approach and the need to combine the results at the end some communication overhead is introduced. The speedup achieved is expressed by Amdahl's law (Wikipedia, 2018b). The term 'embarrassingly parallel' is used to indicate methods that can be easily executed in parallel. Bootstrap sampling is an example of such a method.

From the above one may be tempted to conclude that algorithmic complexity is the only consideration when looking at the speed of an algorithm. It clearly is an important one but other factors also affect the overall speed of an implemented method. The most important other considerations are:

1) The hardware available (especially processor clock frequency, I/O performance of disks, use and number of multiple computers)
2) Any other tasks performed by (other users on) the system used
3) The programming language and compiler used
4) The programming skills of the person writing the code
5) Use of in-memory techniques
6) Use of specialized hardware (such as GPGPU's or dedicated chips)
7) Efficiently combining the factors listed above

This list makes clear that (increasing) the speed by which large amounts of data are processed actually depends on multiple 'components' and not only on the method chosen and the way it is implemented. This makes it challenging to master the 'art' of processing of data in a speedy fashion. However, creating a very fast implementation of a particular method can really help a lot of people and any production processes depending on it. Particular for (near) real-time processes, making use of streaming data, the availability of such implementations are essential.

As an example of the gain that can be obtained, table 11 shows the effect of various implementations in R on the three processing stages, including the effect of parallel processing, on the processing time of a particular subset of road sensor data. From this table it is clear that a considerable increase in processing speed (nearly 20-times) can be achieved by adjusting the code and making use of faster implementations of essential functions needed.

Table 11. Effect of various programming approaches on the processing time of road sensor data in R.

Script nr.	Read-stage	Processing-stage	Write-stage	Time (min)	Speed-up
Script1	read.table	data.frame	write.table	110.9	-
Script2	read.table	data.table	write.table	108.2	1.02x (2.4%)
Script3	fread	data.table	write.table	20.8	5.3x (81%)
Script4	fread	data.table	fwrite	21.1	5.3x (81%)
Script5	fread (parallel 8)	data.table	fwrite	12.8	8.7x (88%)
Script6	fread (parallel 8)	data.table (parallel 8)	fwrite	12.2	9.1x (89%)
Script7	fread (parallel 8)	data.table (parallel 8)	fwrite (parallel 8)	8.6	13.0x (92%)
Script8	zip-> fread (parallel 8)	data.table (parallel 8)	fwrite (parallel 8)	6.7	16.6x (94%)
Script9	zip-> fread ---------->	data.table (parallel 8)	fwrite (parallel 8)	6.1	18.2x (94%)
Script10	zip-> fread ---------->	data.table (parallel 16)	fwrite (parallel 16)	5.7	19.4x (95%)

Explanation of words used. *Script* indicated the various variants of the code created; each scripts reads data, processes it and stores the final result. *read.table* refers to the standard way in which R reads CSV-files. A *data.frame* is the standard way of working with data in memory in R. *write.table* is the standard way of storing CSV-files in R. A *data.table* is a faster implementation of a data.frame in R. *fread* is a fast CSV-read function included in the data.table package. *fwrite* is a user-defined function in C++ for storing CSV-files as fast and compact as possible. *parallel* refers to running the entire process in parallel and the number in brackets refer to the number of cores used. *zip* indicated that the data is read as a zipped CSV-file. The arrow linking fread and data.table in scripts 9 and 10 indicate that the read and processing steps are combined in a single step.

In various WP's of the ESSnet poor performance of analysis is mentioned as a downside to the work performed. WP3 indicates the poor performance of the Hadoop queries on smart meter data as a drawback of this system implemented (WP3 Del 3.2). Spark could be a solution for this (section 2.6). Increasing the processing of smart meter data is also a huge problem in Denmark. In WP7 the rapid processing of satellite images is extremely important (WP7 Del 7.1). By processing each image in 42 smaller chunks this could be achieved. This nicely illustrates the effectiveness of parallel processing. Without such considerations, processing terabytes of road sensor data would also not be achieved (Puts et al., 2016).

3.9.3. Discussion

When working with huge amounts of data it is essential to consider the overall time needed to achieve a particular task. Choosing the appropriate method, implementation and platform are vital to be successful in this. Without it, a particular task may simple not be finished on time. The few examples given for WP's and Road Intensity statistics illustrate this and are -very likely- just the tip of the iceberg of these issues. When a Big Data process start to be formed, it is clear that is essential to find out how much time each step takes. Particularly for streaming data it is essential to assure that the micro-batch of data processed in a particular step is completely processed before the next step starts.

3.9.4. Literature

[1] Almudevar, A. (2014) Approximate Iterative Algorithms. CRS Press, London.

[2] Bachmann, P.G.H. (1894) Analytische Zahlentheorie. Leipzig. For an overview see also the table on https://en.wikipedia.org/wiki/Big_O_notation

[3] Pacheco, P.S. (2011) An Introduction to Parallel Programming. Elsevier, Amsterdam.

[4] Puts, M., Tennekes, M., Daas, P.J.H., de Blois, C. (2016) Using huge amounts of road sensor data for official statistics. Paper for the European Conference on Quality in Official Statistics 2016, Madrid, Spain.
[5] Wikipedia (2018a) Algorithmic efficiency. Located at: https://en.wikipedia.org/wiki/Algorithmic_efficiency
[6] Wikipedia (2018b) Amdahl's law. Page located at: https://en.wikipedia.org/wiki/Amdahl%27s_law
[7] WP3 Del 3.2 (2016) Smart meters: Report on production of statistics: methodology. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_2.
[8] WP7 Del 7.1 (2017) Multi domain: Results of Population, Tourism/Border crossing and Agriculture. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/1/15/WP7_Deliverable_7.1_7.2_7.3_2017_02_01.pdf

3.10. Training/skills/knowledge

3.10.1. Introduction

For Big Data to be used in a statistical office, it is essential that employees are aware of the ways in which these data can be applied in the statistical process, are familiar with the benefits of using Big Data specific IT-environments and possess the skills needed to perform these tasks. In the subsequent section it is assumed that all knowledge needed is (somewhere) available to fulfil these needs. Training is a way to transfer this knowledge to others. However, people can be trained in various ways. Examples are training of NSI-staff in house by Big Data experienced colleagues, training by coaches from a commercial company, such as employees of a Big Data company or experienced Big Data trainers, or by following a training course at an international level, which could be held either on- or offline.

In principle the new skills needed can be contributed to three kind of job profiles, these are: a data scout, a data engineer and a data scientist.

A data scout is a person that focusses on finding new data sources, getting into contact with the organisation producing and/or maintaining the data, and is involved in assuring a steady delivery of data.

A data engineer is a person that build pipelines that collects, processes, transforms and stores data in a way that enables others (usually data scientists) to use it.

A data scientist is a person that analyses and extract information from all kinds of data, in particular new digital data sources, to assist an organization in its decision-making process.

3.10.2. Examples and methods

Examples of international training courses are the Big Data courses included in the European Statistical Training Program (ESTP, 2018), the Big Data lectures included in the European Master of Official Statistics (EMOS, 2018) or a Big Data bachelor or master program at a University or High school (see for instance Reinstein, 2017). In a nutshell, these courses enable participants to get acquainted with Big Data specific methods, techniques and IT-environments. The knowledge is primarily transferred by lecturing and some courses also include a hands-on training component. Since the ESTP trainings are the most relevant for NSI employees these are used as an example. To get an idea of the skills taught, we list the relevant Big Data and Data Science training courses in the ESTP program below including a brief description:

1. Introduction to Big Data and its Tools

Introduction to the concepts of Big Data, the associated challenges and opportunities, and the statistical methods and IT tools needed to make their use effective in official statistics.

2. Can a Statistician become a Data Scientist?

   Demonstration of innovative techniques and their applications, identification of the skills needed for statisticians working at NSI's to test the use of Big Data and other non-traditional sources of data for Official Statistics.

3. Machine Learning Econometrics

   Demonstration of innovative algorithm-based techniques for data analysis, with application to datasets for official statistics as well as for other sources (e.g. Big Data and text data).

4. Hands-on Immersion on Big Data Tools

   Introduction to the state-of-the-art IT tools required to process datasets of large size and test the tools in practices on real-world Big Data sets

5. Big Data Sources – Web, Social Media and Text Analytics

   Apply web scraping and other techniques to collect texts from the web and learn how to analyse and mine them in order to determine their content and sentiment.

6. Automated Collection of Online Prices: Sources, Tools and Methodological Aspects

   Understand the advantages, risks and challenges of automated methods of collecting online prices (web scraping) including methods needed to calculate price indices and learn how to build web scrapers independently.

7. Advanced Big Data Sources – Mobile Phone and Other Sensors

   Learn how to explore, analyse and extract relevant information from large amounts of mobile phone and other sensor data, including its metadata.

In these training courses participants are introduced to topics such as High Performance Computing environments (including Hadoop, Spark and GPGPU's), data cleaning procedures, machine learning methods and ways to collect and analyse various Big Data sources (such as web pages, social media messages, mobile phone data, sensor data and satellite images) with open source software. Each of these topics provides knowledge and form essential building blocks needed for the creation of Big Data based statistics.

During the ESSnet Big Data project a questionnaire has been send out to the WP-leaders to identify the skills needed for the various task included in their WP. In this report these results are combined with the relevant, more general, statistical skills identified in the GSBPM framework (GSBPM, 2017). The results are shown in table 12.

Table 12. Big Data specific skills identified for the production of Big Data based official statistics.

No.	Skills	Tasks involved	Goal
1	Negotiating	Contacting and negotiating with data owners (usually private companies)	-Obtaining access to data owned by private companies -Maintaining good contact to assure a steady delivery of data
2	Infrastructural	a. Setting up and maintaining Big Data IT environments b. Transporting large amounts of data	-Making sure the IT environment keeps functioning -Making sure access to the data is carefully controlled
3	Combining	a. Linking Big Data to other data sources b. Determining population composition c. Identifying linking variables/features	Combining Big Data with other data sources in various ways
4	Checking/editing	Checking and editing large amounts of data in an efficient way	Making sure the data is of the highest quality possible

No.	Skills	Tasks involved	Goal
5	Analysing	a. Dealing with and analyzing large amounts of data b. Programming/using efficient methods c. Asking the right analysis questions	-Assuring large amounts of data are being processed and analyzed within a reasonable time -Assuring the right question is answered by the analysis performed
6	Statistical methods (incl. Machine learning)	a. Developing and applying methods specific for Big Data based estimations b. Applying advanced analytical methods, such as Machine learning, in the most optimal and efficient way	-Making sure reproducible results of the highest quality are produced -Making sure privacy is preserved in the final outcome
7	Visualization	a. Creating clear and interpretable visualizations b. Summarizing data and results	Presenting the results as good as possible

As is clear from table 12, data scouts (skill 1), data engineers (skill 2 and 5) and data scientists (skills 3-7) are needed to bring official statistics to the next level. In addition, it can be expected that the training courses also influence the mindset needed to enable the successful use of Big Data.

The mindset is an important consideration because the paradigm commonly observed in NSI's is usually focused on dealing with sample surveys. In the latter a statistician is used to predominantly look at the way the data is collected (the design), the representativity of the response and the estimation of variance. A similar approach is commonly observed when NSI employees deal with administrative data. Big Data oriented work, in contrast, focusses much more on the population composition and the quality of the data in a source and the potential bias of the estimate derived from it. The latter requires is a considerable change in the way an NSI employee is commonly used to work. Illustrating various ways in which Big Data can be successfully used for official statistics is an important contributor to stimulate such a change. The introduction to Big Data specific IT-environments support this as well because it demonstrates that there is no need to keep working with relative small data sets.

3.10.3. Discussion

Training employees is an important building block in enabling the use of Big Data for official statistics. The skills mostly needed during the work performed in the ESSnet Big Data are listed in table 10. These can all be trained in some way or another. However, one may wonder if simply following a training course is enough? Certainly when a participant is acting at the Big Data forefront compared to the other employees at his/her NSI, following such a course by one or a few employees, does not immediately result in an increase in the production of Big Data based statistics when this person returns. Support by higher management, a certain number of employees with similar goals and skills, the availability of one or more Big Data sources and appropriate privacy protecting regulations are the minimum combination required to initiate this process. Additional contributors to this are a Big Data ready IT-environment and contact with either Universities, research institutes or other NSI's with expertise on the topic studied. The latter can also be achieved by involvement in an international Big Data project, such as the ESSnet Big Data.

3.10.4. Literature

[1] EMOS (2018) European Master in Official Statistics. Located at: http://ec.europa.eu/eurostat/web/european-statistical-system/emos

[2] ESTP (2018) European Statistical Training Programme. Located at: http://ec.europa.eu/eurostat/web/european-statistical-system/training-programme-estp
[3] GSBPM (2017) Statistical Training Framework based on the GSBPM. Report located at: https://www.unece.org/fileadmin/DAM/stats/documents/ece/ces/ge.58/2017/mtg4/Paper_6_Training_based_on_the_GSBPM_Ireland.pdf
[4] Reinstein, I. (2017) Best Masters in Data Science and Analytics – Europe Edition. KDnuggets, located at: https://www.kdnuggets.com/2017/12/best-masters-data-science-analytics-europe.html

4. Conclusions

As mentioned in the document, there are several IT aspects that have already been explored in different ESSNet Big Data projects. This experience allows us avoiding common problems regarding planning and developing further Big Data projects. On the other side, lots of IT aspects still needs to be evaluated. That's why this document contains suggested tools and methods for Big Data projects and don't cover all the aspects regarding IT.

Current knowledge allowed to formulate original conclusions on how Big Data IT infrastructure can be used in official statistics. It was possible by exploring ten different aspects of Big Data, including Big Data processing life cycle, metadata management (ontology), format of Big Data processing, data-hub and data-lake, data source integration, choosing the right infrastructure, list of secure and tested API's, shared libraries and documented standards, speed of algorithms and training/skills/knowledge.

We hope this document will be a good support for any further Big Data projects that can be conducted in official statistics.

5. Abbreviations and acronyms

API – Application Programming Interface
AWS – Amazon Web Services
CBS – Centraal Bureau voor de Statistiek (Netherlands)
CSS – Cascade Style Sheet
CSV – Comma Separated Values file format
COTS – Commercial off-the-shelf
EMOS – European Master in Official Statistics
ESS – European Statistical System
ESSnet - A network of several ESS organisations aimed at providing results that will be beneficial to the whole ESS
ESTP - European Statistical Training Program
GPGPU - General-Purpose computing on Graphics Processing Units
GPU - Graphics Processing Units
GSBPM – Generic Statistical Business Process Model
GSIM – Generic Statistical Information Model
GAMSO – Generic Activity Model for Statistical Organizations
HTML – Hypertext Markup Language
IT – Information Technology
JSON – JavaScript Object Notation
NLP – Natural Language Processing
NoSQL – non relational database (non SQL)
NSI – National Statistical Institute
ONS – Office for National Statistics (UK)
SVN – SubVersioN
WP - Workpackage
XML – Extensible Markup Language

6. List of figures and tables

Figure 1. Decision process of using the format of data processing 17
Figure 2. Big Data Processing Path 22
Figure 3. Statistical Big Data Processing Path..... 22
Figure 4. Relational Data Processing Path 22
Figure 5. Data Integration using a Data Bus 23
Figure 6. Data Integration using a Connector 23
Figure 7. Data integration using Data Virtualization 24
Figure 8. Conceptual Big Data Platform 26
Figure 9. Typical structure of the project in GitHub repository 35
Figure 10. An example of GitHub clone process 36
Figure 11. Big O complexity chart of algorithms. The number of operations are shown versus the number of elements (size n) for a number of complexity functions. (from http://bigocheatsheet.com/) 38

Table 1. Core principles of metadata management of the Common Metadata Framework 12
Table 2. Metadata quality evaluation on WP4 AIS Data 13
Table 3. Main features of data processing by selected Big Data software 16
Table 4. Data processing examples depending on the data used 17
Table 5. Data Integration use cases in WPs..... 21
Table 6. Comprehensive list of what is being used across the ESSnet on Big Data 27
Table 7. Brief overview of API's 32
Table 8. Main features of selected source code repositories 34
Table 9. Popular GitHub repositories for official statistics..... 36
Table 10. An overview of the various algorithmic complexity notations discerned 38
Table 11. Effect of various programming approaches on the processing time of road sensor data in R. 40
Table 12. Big Data specific skills identified for the production of Big Data based official statistics. 42

7. Literature

1. https://statswiki.unece.org/display/GSBPM/Issue+%2322%3A+Metadata+Management+-+GSBPM+and+GAMSO [as of 5.01.2018].
2. A Suggested Framework for the Quality of Big Data, Deliverables of the UNECE Big Data Quality Task Team, December, 2014, https://statswiki.unece.org/display/bigdata/2014+Project?preview=%2F108102944%2F108298642%2FBig+Data+Quality+Framework+-+final-+Jan08-2015.pdf [as of 4.01.2018].
3. Almudevar, A. (2014) Approximate Iterative Algorithms. CRS Press, London.
4. Bachmann, P.G.H. (1894) Analytische Zahlentheorie. Leipzig. For an overview see also the table on https://en.wikipedia.org/wiki/Big_O_notation
5. Beyer M. A., Thoo E., Selvage M. Y. and Zaidi E. (2017): Magic Quadrant for Data Integration Tools. Gartner report, August version, ID Number: G00314940
6. Centre of Excellence on Data Warehouse (2017): Statistical Data Warehouse Design Manual – Architecture Chapter. EUROSTAT available on CROS Portal https://ec.europa.eu/eurostat/cros/content/s-dwh-design-manual_en
7. Chisholm, M. (2008) Six Architectural Styles of Data Hubs. Located at: http://www.b-eye-network.com/view/8109
8. Demunter, C., Seynaeve, G. (2017) Better quality of mobile phone data based statistics through the use of signaling information – the case of tourism statistics, NTTS Conference, 13-17 March 2017.
9. Dijcks J. P. and Gubar M. (2014): Integrating SQL and Hadoop. Business Intelligence Journal, Vol. 19, No. 2.
10. EMOS (2018) European Master in Official Statistics. Located at: http://ec.europa.eu/eurostat/web/european-statistical-system/emos
11. ESTP (2018) European Statistical Training Programme. Located at: http://ec.europa.eu/eurostat/web/european-statistical-system/training-programme-estp
12. Feldman, D. (2018) Data Lakes, Data Hubs, Federations: Which One Is Best? Blog available at: https://www.marklogic.com/blog/data-lakes-data-hubs-federation-one-best/
13. GSBPM (2017) Statistical Training Framework based on the GSBPM. Report located at: https://www.unece.org/fileadmin/DAM/stats/documents/ece/ces/ge.58/2017/mtg4/Paper_6_Training_based_on_the_GSBPM_Ireland.pdf
14. GSPBM (2013) Generic Statistical Business Processing Model version 5.0. Located at: https://statswiki.unece.org/display/GSBPM/GSBPM+v5.0
15. http://flightaware.com/commercial/flightxml/pricing_class.rvt, accessed 9th of November 2017
16. Kepner J (2017).: Enabling Scale-Up, Scale-Out, and Scale-Deep for Big Data. MIT Lincoln Laboratory. Supercomputing center
17. Kune R. ET AL.(2016): The anatomy of big data computing. Published online 9 October 2015 in Wiley Online Library DOI: 10.1002/spe.2374
18. Lehman D., Fekete D. and Vossen G.: Technology Selection for Big Data and Analytical Applications. European Research Center for Information Systems (ERCIS) Working Paper No. 27
19. NIST Big Data Public Working Group (2015) NIST Big Data Interoperability Framework: Volume 1, Definitions. Final version. http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1500-1.pdf
20. Olavsrud, T. (2017) 3 keys to keeping your data lake from becoming a data swamp. Available at: https://www.cio.com/article/3199994/big-data/3-keys-to-keep-your-data-lake-from-becoming-a-data-swamp.html
21. Pacheco, P.S. (2011) An Introduction to Parallel Programming. Elsevier, Amsterdam.
22. Peng, R., Matsui, E. (2016) The Art of Data Science, A guide for anyone who works with data. Leanpub book, lulu.com
23. Phule R. and Ingle M. (2013): A Survey on Scalable Big Data Analytics Platform. International Journal of Science and Research (IJSR) ISSN (Online): 2319-7064
24. Puts, M., Tennekes, M., Daas, P.J.H., de Blois, C. (2016) Using huge amounts of road sensor data for official statistics. Paper for the European Conference on Quality in Official Statistics 2016, Madrid, Spain.
25. Puts, M., Tennekes, M., Daas, P.J.H., de Blois, C. (2016) Using huge amounts of road sensor data for official statistics. Paper for the European Conference on Quality in Official Statistics 2016, Madrid, Spain.
26. Reinstein, I. (2017) Best Masters in Data Science and Analytics – Europe Edition. KDnuggets, located at: https://www.kdnuggets.com/2017/12/best-masters-data-science-analytics-europe.html
27. Singh D. and Reddy C.K. (2015): A Survey on Platforms for Big Data Analytics. Journal of Big Data, Springer 2015

28. Statistics Netherlands (2015) A first for Statistics Netherlands: launching statistics based on Big Data. Located at: https://www.cbs.nl/NR/rdonlyres/4E3C7500-03EB-4C54-8A0A-753C017165F2/0/afirstforlaunchingstatisticsbasedonbigdata.pdf.
29. Stein, B., Morrison, A. (2014) The enterprise data lake: Better integration and deeper analytics. Tech. Forecast 1, pp. 1-9.
30. Steiner J. (2011): Managing Big Data by Using Hadoop and Oracle Exadata. Oracle White Paper, 2011
31. UNECE (2016) Sandbox. Link: https://statswiki.unece.org/display/bigdata/Sandbox.
32. Wikipedia (2018a) Algorithmic efficiency. Located at: https://en.wikipedia.org/wiki/Algorithmic_efficiency
33. Wikipedia (2018b) Amdahl's law. Page located at: https://en.wikipedia.org/wiki/Amdahl%27s_law
34. WP2 Del 2.2 (2017) Methodological and IT issues and Solutions. Chapter 3 "Description of a Reference Framework for Web Scraping of Enterprises Web Sites". Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/6/66/WP2_Deliverable_2.2_2017_07_31.pdf
35. WP3 Del 3.1 (2016) Smart meters: Report on Data access and Data handling. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_1.
36. WP3 Del 3.2 (2016) Smart meters: Report on production of statistics: methodology. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_2.
37. WP3 Del 3.2 (2016) Smart meters: Report on production of statistics: methodology. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/index.php/WP3_Report_2.
38. WP4 Del 4.3 (2018) Report about sea traffic analysis using AIS-data. Located at; https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/5/5c/WP4_Deliverable_4.3_2017_07_21_v1.0.pdf
39. WP7 Del 7.1 (2017) Multi domain: Results of Population, Tourism/Border crossing and Agriculture. Located at: https://webgate.ec.europa.eu/fpfis/mwikis/essnetbigdata/images/1/15/WP7_Deliverable_7.1_7.2_7.3_2017_02_01.pdf